- MVA RESEARCH INTERNSHIP -

Planning with JEPA world models and value functions

Matthieu DESTRADE

8th of April, 2025 - 23rd of August, 2025

CILVR, Courant Institute, NYU

Internship tutors: Dr. Yann LeCun (NYU) and Dr. Jean Ponce (ENS, NYU)

Contents

1 Acknowledgments

I would like to express my deepest gratitude to my two advisors, Dr. Yann LeCun and Dr. Jean Ponce, who oversaw my internship at NYU and provided valuable guidance and advice for my research.

I am also grateful to Dr. Oumayma Bounou and Dr. Quentin Le Lidec for our enriching discussions on the topics addressed in this report and for their constant availability and support.

Finally, I would like to thank Vlad Sobal and Kevin Zhang, who kindly shared their implementation of JEPA models for planning, which served as the basis for my own code.

2 Abstract

In order to obtain a deep learning model capable of "reasoning," it appears necessary to train a world model, that is, a model that captures the laws governing the evolution of a dynamical system. Joint-Embedded Predictive Architectures (JEPA) provide one way of implementing such models. They learn representations of the world along with a predictor, using prediction loss as the training criterion. Beyond providing access to meaningful representations of an environment's states, these models also enable action planning, which is the task of optimizing a sequence of actions to reach a goal from a given initial state.

In this study, we propose two approaches to improve the action planning capabilities of JEPA world models. The first consists in learning representations such that the distance between two state representations reflects the goal-conditioned value function associated with a reaching cost. We describe a method to achieve this and implement it, obtaining better planning performance than traditional JEPA models on simple planning tasks.

The second approach is to use a hierarchy of representations to better capture relationships between distant states and to increase the prediction horizon. Although we do not obtain good results with this method, we highlight an issue that arises when performing action planning with a JEPA model: optimizing action representations often yields unrealistic solutions on which the predictor is unreliable. To address this issue, we propose regularizing the action representation space. However, this does not lead to improved planning results on our tasks.

3 Introduction

World models are a class of deep learning architectures designed to capture the dynamics of general systems: the "world" [Din+25]. They are currently regarded as one of the most promising candidates for the emergence of artificial general intelligence (AGI). Unlike large language models, with which they are often compared, world models are explicitly trained to learn the laws that govern a system's behavior. As a result, they can predict the system's future evolution and make decisions accordingly. More specifically, world models typically have two main objectives: to learn effective representations of a system's states and to accurately predict its future evolution.

Since the definition of a world model is quite broad, numerous architectures have been proposed to implement them. One such approach is the Joint-Embedded Predictive Architecture (JEPA) [LC22]. This approach is based on the idea that representations of successive states of a system should be learned in such a way that the representation of the next state can be predicted from that of the current state. To do so, it leverages three models: an encoder of states, an encoder of actions and a predictor. Several works have explored the use of such models for representation learning [Ass+23; Gar+24; Bar+24; Ass+23] and for action planning [Sob+25; Zho+25]. Action planning refers to the task of optimizing a sequence of actions that allows one to reach a desired goal from an initial state of a system.

In this work, we aim at improving the current capabilities of JEPA world models for action planning. To this intent, we identify two main avenues for improvement.

The first approach is to leverage the value function as a cost for planning. We draw inspiration from works in the reinforcement learning field, which aim to learn representations of environment states such that the Euclidean distance between them corresponds to the goal-conditioned value function associated with a goal-reaching cost [PKL24; Par+24]. We hypothesize that similar approaches could be applied to JEPA world models, potentially facilitating planning optimization by removing local minima from the planning cost. We also explore the possibility of using a quasidistance rather than a distance in the representation space, as initially studied in [Wan+23]. This is motivated by the fact that the goal-conditioned value function satisfies the properties of a quasidistance but not of a true distance, since it is generally not symmetric. We implement these approaches and compare their planning performance to that of models trained with prediction losses. Our results on standard planning tasks indicate that learning a value function effectively leads to improved planning outcomes.

The second approach we consider to improve the planning capabilities of JEPA world models is the use of hierarchical architectures. Such approaches have already been explored to increase the maximum prediction horizon of JEPA world models [LC22] and to improve the learning of goal-conditioned value functions [Par+24]. We propose a similar approach, training different levels of JEPA models on sequence of states and actions subsampled with increasing strides. We hypothesize that doing so makes distant states appear closer to the models in high levels of the hierarchy, thereby helping them capture relationships between these distant states. We distinguish between two types of hierarchies within the hierarchical JEPA world model. The first is a hierarchy of states, which specifically allows better access to information about distant states in the planning cost. The second is a hierarchy of actions, which specifically enables prediction and planning over longer time frames. We implement this approach and test it on standard plannning tasks. While the hierarchy of states leads to small improvement of planning capabilities, we do not manage to improve our results with the hierarchy of actions.

We identify an issue that arises when performing action planning with JEPA world models, which may explain the poor results obtained with the action hierarchy. In this setting, we are effectively optimizing the input of a deep learning network: a sequence of actions fed into the predictor. The result might therefore be an adversarial example, that is to say sequences of actions that may be unrealistic and on which the predictor's behavior is unreliable. This effect is further exacerbated by the use of hierarchies, since optimization is no longer performed in the original action space but rather in a representation space. To address this issue, we take inspiration from variational autoencoders and regularize the action representation spaces so that any representation can be mapped back to valid actions. While this approach did not yield strong results in our experiments, we believe that further research could refine it, and that similar methods will be necessary for effective hierarchical action planning.

4 Background

Our study focuses on two main research fields: JEPA world models and offline goal-conditioned value function learning.

4.1 JEPA world models

World models are deep learning-based architectures with two main goals, according to [Din+25]:

- Learn representations of the world.
- Predict future states of the world, based on an initial one and actions.

They are inspired by the theory of "mental models" [Joh83], which suggests that humans understand the world by abstracting it and creating relationships between representations of its states. JEPA world models are a way of implementing them for representation learning and action planning, introduced in [LC22]. They notably originate from the following ideas:

- Predicting future states of a system influenced by actions might be easier in a representation space rather than in the original observation space.
- Learning representations such that future states of the environment can be predicted is a good approach to extract relevant information from observations. This is a way to avoid using reconstruction-based losses that tend to focus on irrelevant information for perception [BL24]. This differentiates JEPA models from other reconstruction-based world model architectures, such as Dreamer [Haf+24].

JEPA world models have been used for self-supervised representation learning in a series of works. They have been applied to images, with I-JEPA[Ass+23] and IMW [Gar+24], and videos, with V-JEPA [Bar+24; Ass+23]. For these works, the actions considered to learn the predictor are augmentations of the input data. The quality of the representations learned is confirmed in these works, as well as in further studies such as [Gar+25], which shows that they exhibit basic physics understanding.

A well-trained world model allows one to access a differentiable approximation of the dynamics of an environment, which can be used for a number of tasks and notably optimal control. Optimal control, or action planning, refers to a class of problems in which the goal is to optimize control inputs, or actions, that influence a dynamical system in order to minimize a cost. In this study, we will specifically focus on goal-reaching tasks, in which the cost is related to the distance between the current state and a target state.

Several works have focused on using JEPA world models to solve this kind of tasks. Some of them learn the model from scratch on environments of interest, such as PLDM [Sob+25], while others choose to leverage pre-existing representations and only learn a predictor, such as DINO-WM [Zho+25]. DINO-WM is based on a self-supervised representation learning model, DINO [Oqu+24], whose training is similar to that of JEPA representation learning architectures. Such methods show promising results, but still fail to perfectly solve simple planning tasks.

In the case of our study, we will apply world models to a setting similar to that of offline reinforcement learning, as in [Sob+25] and [Zho+25]. We want to learn an encoder and a predictor using examples of trajectories of an agent in a given environment. We will describe how to train a JEPA world model to learn representations and perform action planning in this setting. To do so, we differentiate between the different parts of a JEPA world model of subsampling stride k in $\mathbb{N}$:

- A state encoder: $\mathcal{E} : \mathrm{S}_0 \to \mathrm{S}_1$, where S_0 is the original observation space and S_1 the state representation space.
- An action encoder: $\mathcal{A} : \mathrm{A}_0^k \to \mathrm{A}_1$, where A_0 is the original action space and A_1 the action representation space. $\mathcal{A}$ takes as input a sequence of k concatenated actions.
- A predictor: $\mathcal{P} : \mathrm{S}_1 \times \mathrm{A}_1 \to \mathrm{S}_1$.

We define a world model as: $\mathcal{W} = (\mathcal{E}, \mathcal{A}, \mathcal{P}, k)$. Typically, $k = 1$ and $\mathcal{A} = \mathrm{Id}$. All spaces considered are subsets of $\mathbb{R}^n$ for appropriate integers n.

4.1.1 Training a world model

Let $T \in \mathbb{N}$. Let us define a trajectory as a sequence of observed states along with the corresponding actions: $((s_t)_{t\in[\![0,T]\!]}, (a_t)_{t\in[\![0,T-1]\!]}) \in \mathrm{S}_0^{T+1} \times \mathrm{A}_0^T$.

A world model is trained using a dataset $\mathcal{D}$ of such trajectories of length T. Using this dataset, the core idea is to learn networks such that we can predict a future state of the environment from an initial state and an action. To do so, we learn the parameters θ which parameterize $\mathcal{E}_\theta$, $\mathcal{A}_\theta$ and $\mathcal{P}_\theta$, so that they minimize the prediction loss on the trajectories:

$$\forall((s_t),(a_t)) \in \mathcal{D},\ \mathcal{L}^\theta_{\text{pred}}((s_t),(a_t)) = \sum_{t=0}^{\lfloor T/k \rfloor - 1} \|\mathcal{P}_\theta(\mathcal{E}_\theta(s_{kt}), \mathcal{A}_\theta((a_j)_{j\in[\![kt,k(t+1)-1]\!]}) - \mathcal{E}_\theta(s_{k(t+1)})\|_2^2 \quad (1)$$

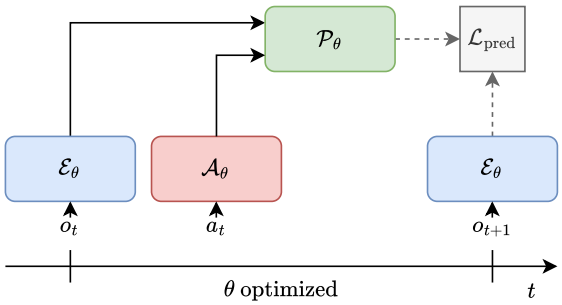


Figure 2: Diagram of a world model at train time for $k = 1$

However, in practice this leads to the collapse of the representations. Indeed, the easiest way for the model to minimize this loss is to make all representations equal and the predictor constant. In order to prevent this from happening, several practical tricks exist:

- Using a VCReg loss [BPL22], as done in [Sob+25]: this loss prevents collapse by forcing the representations $(\mathcal{E}_\theta(s_{kt}))_{t\in[\![0,\lfloor T/k \rfloor]\!]}$ to be well spread accross the representation space.
  Let $N \in \mathbb{N}$, $d \in \mathbb{N}$. The VCReg loss is composed of several terms, and is applied to a batch of vectors of $\mathbb{R}^d$. The first term V of the loss incites the variance of every component of the vectors to be equal to a real scalar γ (typically $\gamma = 1$) over the batch:

$$\forall(z_n)_{n\in[\![1,N]\!]} \in (\mathbb{R}^d)^N, V((z_n)) = \frac{1}{d}\sum_{j=1}^{d} \max\left(0, \gamma - \sqrt{\mathrm{Var}((z_n^j)) + \epsilon}\right) \quad (2)$$

  ,where ϵ is a small real value used for stability, and: $\forall n \in [\![1,N]\!], (z_n^j)_{j\in[\![1,d]\!]} = z_n$.
  The second term C makes it so that the covariance of the coordinates of the vectors is equal to 0 over the batch, and contributes to unentangling the dimensions:

$$\forall(z_n)_{n\in[\![1,N]\!]} \in (\mathbb{R}^d)^N, C((z_n)) = \frac{1}{d}\sum_{i\neq j}(\mathrm{Cov}((z_n))_{i,j})^2 \quad (3)$$

  The loss $\mathcal{L}^\theta_{\text{VCReg}}$ is applied to batches of size N from $\mathcal{D}$ and is defined as:

$$\forall(((s_t^n),(a_t^n))_n) \in \mathcal{D}^N,\ \mathcal{L}^\theta_{\text{pred}}(((s_t),(a_t))_n) = \frac{1}{N}\sum_{n=1}^{N}(\alpha V((\mathcal{E}_\theta(s_t^n))_t) + \beta C((\mathcal{E}_\theta(s_t^n))_t)) \quad (4)$$

$$+\frac{1}{T+1}\sum_{t=0}^{T}(\gamma V((\mathcal{E}_\theta(s_t^n))_n) + \delta C((\mathcal{E}_\theta(s_t^n))_n) \quad (5)$$

  , where $\alpha, \beta, \gamma, \delta$ are positive weights. This loss can be interpreted as a way of maximizing the information preserved in the representations [Shw+24].

- Using an exponential moving average (EMA) scheme during training, as used in [Ass+23; Bar+24; Ass+23]. This consists in using two sets of models during training: $(\mathcal{E}_\theta, \mathcal{A}_\theta, \mathcal{P}_\theta)$ and the target encoder $\mathcal{E}_{\theta_{\text{EMA}}}$. The parameter θ is learned by performing gradient steps on the adapted prediction loss

$$\forall((s_t),(a_t)) \in \mathcal{D},\ \mathcal{L}^\theta_{\text{predEMA}}((s_t),(a_t)) = \sum_{t=0}^{\lfloor T/k \rfloor - 1} \|\mathcal{P}_\theta(\mathcal{E}_\theta(s_{kt}), \mathcal{A}_\theta((a_j)_{j\in[\![kt,k(t+1)-1]\!]}) - \mathcal{E}_{\theta_{\text{EMA}}}(s_{k(t+1)})\|_2 \tag{6}$$

  The parameter θ_{EMA} does not participate in gradient computations, but is updated at every optimization step using: $\theta_{\text{EMA}} \leftarrow \tau\,\theta_{\text{EMA}} + (1-\tau)\theta$, where the momentum τ belongs to $[0,1]$.
  This leads to good results in practice but is not well understood theoretically.

4.1.2 Planning with a world model

We are interested in solving goal-reaching problems. We wish to obtain a sequence of actions that allows an agent to go from a state defined by an observation o in S_0 to a goal g in S_0.

Let $(n_{\text{step}}, T, n) \in \mathbb{N}^3$. Once a world model is trained, it is possible to use a model predictive control (MPC) [GPM89] procedure to optimize actions. It depends on the number of steps n_{step}, on the horizon T and on the number n of actions to be applied at every step.

To perform MPC, one starts by solving the following control problem:

$$\begin{aligned} \underset{(a_t)_{t\in[\![0,T-1]\!]} \in \mathrm{A}_1^T}{\text{minimize}} \quad & \mathcal{L}_{\text{plan}} = \sum_{t=1}^{T} \|s_t - \mathcal{E}(g)\|_2 \\ \text{such that} \quad & \begin{cases} s_0 = \mathcal{E}(o) \\ \forall t \in [\![1, T-1]\!],\ s_{t+1} = \mathcal{P}(s_t, a_t) \end{cases} \end{aligned} \tag{7}$$

The first n actions of the resulting optimized sequence of actions are then applied in the environment to obtain a new starting state. The procedure is repeated with this new state. The optimized actions can also be used as an initialization for the following planning computations. This is done n_{step} times or until the goal is reached.

The optimization of the actions can be done with different approaches:

- Gradient descent. Since $\mathcal{E}$, $\mathcal{A}$ and $\mathcal{P}$ are differentiable, it is possible to minimize the loss $\mathcal{L}_{\text{plan}}$ by performing gradient descent with respect to the actions $(a_t)_{t\in[\![0,T-1]\!]}$.
- Cross Entropy Minimization (CEM) [Zho+25]: This consists in computing the costs $\mathcal{L}_{\text{plan}}$ for several trajectories that are obtained by randomly sampling action sequences of mean (a_t) (randomly initialized) and standard deviation σ. An optimization step then consists in computing the mean and standard deviation of a chosen number of the action sequences with lowest cost. New actions can then be sampled with the resulting mean and standard deviation, and the procedure can be repeated several times. It eventually outputs an action sequence equal to the last mean computed.
- Model Predictive Path Integral (MPPI) [WAT15]. Let $N \in \mathbb{N}$. This consists in computing the costs $(\mathcal{L}_{\text{plan}_n})_{n\in[\![1,N]\!]}$ for trajectories that are obtained by summing an initial action sequence (a_t) with perturbations $((\delta a_{t,n}))_{n\in[\![1,N]\!]}$. An optimization step of the initial sequence is then:

$$(a_t) \leftarrow \left(a_t + \frac{\sum_{n=0}^{N} w_n \delta a_{t,n}}{\sum_{n=1}^{N} w_n} \right) \tag{8}$$

  , where: $\forall n \in [\![1,p]\!], w_n = e^{-\mathcal{L}_{\text{plan}_n}/\lambda}$, and λ is a non-zero temperature parameter.
  This is repeated through the MPC, using the actions output at a step of the procedure as initialization for the next step.

These optimization methods have different drawbacks and advantages. While MPPI and CEM are efficient in small dimension, they might struggle to explore the optimization space if it is of high dimension, and might not be able to output a convincing result. Gradient descent is less sensitive to these problems but can easily converge to bad local minima if the landscape of the cost is not smooth enough. The authors of [Flo+21] notice these issues, and propose to impose a penalty on the gradient of the networks with respect to their inputs to improve gradient descent performances.

In addition to these optimization problems that can harm planning results, MPC is limited by the fact that it has a finite horizon, and can struggle to optimize actions for distant goals. Some of these issues can be reduced by using a value function.

4.2 Learning a value function

To improve the effectiveness of MPC, some works have proposed taking inspiration from reinforcement learning, and learning a value function to guide the MPC procedure [Jor+25; Law+25]. It indeed allows to account for longer time horizons than the ones used during MPC, and can stabilize the procedure by providing an additional cost term whose minimization facilitates goal-reaching tasks.

In this work, we will only consider deterministic environments.

4.2.1 Goal-conditioned value function

We are interested in learning a value function without any reward information in our training dataset, and that can be used for goal-reaching tasks. We are therefore in the goal-conditioned reinforcement learning setting [Sch+15].

Let us define the goal-conditioned discounted value function $V^\star : \mathrm{S}_0^2 \to \mathbb{R}_- \cup \{-\infty\}$:

$$V^\star(s,g) = \lim_{T\to+\infty} \sup_{(a_t)_{t\in[\![0,T]\!]}\in \mathrm{A}_0^{T+1}} \sum_{t=0}^{T} -\gamma^t C(s_t,a_t,g) \tag{9}$$

$$\text{with} \begin{cases} s_0 = s \\ \forall t \in [\![0, T-1]\!], s_{t+1} = d(s_t,a_t) \end{cases} \tag{10}$$

, where $d : \mathrm{S}_0 \times \mathrm{A}_0 \to \mathrm{S}_0$ encodes the dynamics of the environment, $\gamma \in [0,1]$ is the discount factor, and $C : \mathrm{S}_0 \times \mathrm{A}_0 \times \mathrm{S}_0 \to \mathbb{R}_+$ is a positive cost.

This value function satisfies the Bellman equation:

$$\forall (s,g) \in \mathrm{S}_0^2, V^\star(s,g) = \sup_{a\in\mathrm{A}_0} \left(-C(s,a,g) + \gamma V^\star(d(s,a),g)\right) \tag{11}$$

If $\gamma < 1$, the unique solution to this equation is the value function $V^\star$.

We are interested in learning a function that satisfies Bellman's equation and that could be used as a value function to plan actions.

4.2.2 Implicit Q-learning (IQL)

Training a model to learn a value function with trajectories of random or expert agents in a given environment is the setting of offline reinforcement learning [Lev+20]. Several works have studied methods to learn a value function from such data. One of the main difficulties to do so is that training trajectories are potentially suboptimal: they do not always correspond to the shortest path between the points they pass through. Training a simple regression model that would assume that the goal-conditionned value function is the number of time steps between two states is therefore ineffective to learn a value function.

A classical approach to learning the goal-conditioned value function is implicit Q-learning (IQL) [GBL23] and [KNL21]. It deals with the issue previously mentioned by using expectile regression. Its original formulation involves learning an action-value function on top of the value function, but we will focus on a variant of this method developed by [Xu+23] that does not require the action-value function and only learns the

goal-conditioned value function. This makes it possible to learn a value function with datasets of trajectories that only contain state information, in cases in which the cost we consider does not depend on actions.

The IQL method consists in training a network V_θ such that it minimizes the following risk:

$$\mathcal{R}^\theta_{\text{IQL}} = \mathbb{E}_{s\sim \text{S}_0, a\sim\pi(s), g\sim \text{S}_0}\left[L^2_\tau(-C(s,a,g) + \gamma V_{\bar{\theta}}(d(s,a),g) - V_\theta(s,g))\right] \tag{12}$$

where π is the training policy. The weights $\bar{\theta}$ are those of the "target" network, and they are not used to compute the gradient. They can correspond to a stop gradient or an EMA procedure.

The function $L^2_\tau : \mathbb{R} \to \mathbb{R}$ with τ in $[0,1]$ allows to perform an expectile regression. It is defined by:

$$\forall x \in \mathbb{R}, L^2_\tau(x) = |\tau - \mathbb{1}(x<0)|x^2 \tag{13}$$

4.2.3 Convergence of IQL

Let us prove that using the previously introduced loss allows one to learn the goal-conditioned value function.

a) Reasoning behind the choice of risk

The core idea behing the choice of loss is the following theorem [KNL21]:

Theorem 1. *Let us define*

$$\forall \tau \in]0,1[, m_\tau := \arg\min_{m_\tau} \mathbb{E}_{x\sim X}(L^2_\tau(x - m_\tau)) \tag{14}$$

where X is a real-valued random variable with a bounded support. This means that there exists a finite interval I contained in $\mathbb{R}$ such that $\mathbb{P}(X \in I) = 1$.

We have:

$$m_\tau \xrightarrow[\tau\to 1]{} x^+ \tag{15}$$

where x^+ is the supremum of the support of X.

The support of X is the smallest set S of values (in the sense of set inclusion) such that $\mathbb{P}(X \in S) = 1$. This theorem means that performing expectile regression allows to access to the supremum of the values taken by a real random variable when τ tends to 1. We propose in the Appendix 8.1 a complete proof of this result.

An example of an application of this theorem is displayed in Fig. 3, in the case where L^2_τ is applied to a linear regression task. The parameters are optimized so that the regression line approximates the upper hull of the set of points when τ is close to 1.

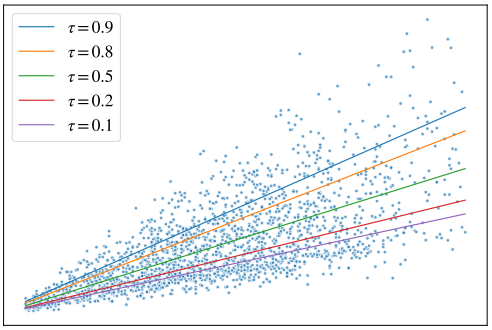


Figure 3: Visualization of expectile regression for different values of τ

Let $\tau \in]0,1[$, $\gamma \in]0,1[$ and π be a training policy, that is to say a function that assigns to each state in S_0 the distribution over actions used to build the training dataset. It is a standard result that the fixed point of the Bellman operator is the value function $V^\star$ [SB18, Section 3.7]. This operator is defined by:

$$\forall V : \mathrm{S}_0^2 \to \mathbb{R}_- \cup \{-\infty\}, \forall (s,g) \in \mathrm{S}_0^2, (\mathcal{T}^\star V)(s,g) = \sup_{a \in \mathrm{A}_0} \left(-C(s,a,g) + \gamma V(d(s,a),g)\right) \tag{16}$$

Taking inspiration from the expectile regression result, we consider the Bellman expectile operator defined as:

$$\forall V : \mathrm{S}_0^2 \to \mathbb{R}_- \cup \{-\infty\}, \forall (s,g) \in \mathrm{S}_0^2, (\mathcal{T}_\tau^\pi V)(s,g) = \arg\min_v \mathbb{E}_{a \sim \pi(s)}[L_\tau^2(-C(s,a,g) + \gamma V(d(s,a),g) - v)] \tag{17}$$

It is worth noting that two different quantities related to value functions appear. The first, $V(d(s,a),g)$, is what we call the target value function, and is used to guide the optimization. The second, v, corresponds to the value function that is actually optimized. The use of a stop gradient in $\mathcal{L}_{\mathrm{IQL}}$ stems from this.

Intuitively, it is possible to approximate the Bellman expectile operator by computing the minimum argument with a gradient descent procedure. For τ sufficiently close to 1, this operator should be close to the optimal Bellman operator and allow one to access to $V^\star$

b) A rigorous proof for an adapted operator

Since we cannot solve the minimization problem exactly, we consider the one-step gradient operator with gradient step α in $]0,1[$, which approximates the previous operator (it corresponds to one step of a gradient descent optimizing v, which is initially set to $V(s,g)$):

$$\forall V : \mathrm{S}_0^2 \to \mathbb{R}_- \cup \{-\infty\}, \forall (s,g) \in \mathrm{S}_0^2, ((\mathcal{T}_g)_\tau^\pi V)(s,g) = V(s,g) + 2\alpha \mathbb{E}_{a \sim \pi(s)}[L_\tau(-C(s,a,g) + \gamma V(d(s,a),g) - V(s,g))] \tag{18}$$

where: $L_\tau(x) = |\tau - \mathbb{1}(x < 0)|x$.
For convenience we will denote by $\mathcal{T}_\tau^\pi$ this gradient operator $(\mathcal{T}_g)_\tau^\pi$.

The authors of [Ma+21] propose a proof that this operator allows one to obtain $V^\star$. They first prove that $\mathcal{T}_\tau^\pi$ is a γ_τ-contraction operator with

$$\gamma_\tau = 1 - 2\alpha(1-\gamma)\min\{\tau, 1-\tau\} \tag{19}$$

We denote by $V_\tau^\star$ the fixed point of this operator (which exists for since $\gamma_\tau < 1$). The authors of [Ma+21] finally prove that:

$$\lim_{\tau \to 1} V_\tau^\star = V^\star \tag{20}$$

The proofs are described in the Appendix B of [Ma+21].

c) Link to the loss $\mathcal{L}_V$

When optimizing $\mathcal{L}_V$, one gradient descent step approximates one application of $\mathcal{T}_\tau^\pi$ for the chosen τ and the π used to build the training dataset. The main sources of inaccuracy during this approximation are:

- The fact that the expressivity of the network V_θ is limited. It is not possible to optimize V for every couple (s,g) of S_0^2 independently. This might be mitigated by using a sufficiently expressive network.
- The fact that the number of training samples does not span the entirety of S_0^2. It is not possible to optimize V for every possible value of (s,g). This might be mitigated by using large batch sizes, or using the same target network for several optimization steps.

If we suppose that these inaccuracies are negligible, the global gradient descent process allows to converge to the fixed point of $\mathcal{T}_\tau^\pi$, which is expected to be close to $V^\star$ if τ is close to 1.

Therefore, learning a value function through the IQL approach is unbiased for deterministic environments. However, it is not the case for stochastic environments, for which this procedure tends to overestimate the value function, because it interprets uncontrollable variations of the environment as the result of the application of an action [Par+24].

4.2.4 Hierarchical representations

Several works have observed that learning a goal-conditioned value function in the offline setting is difficult, especially when dealing with distant goals. In order to tackle this issue, one possible approach is to introduce a hierarchy of models learning the goal-conditioned value function at different scales [Par+24; Gup+19]. This idea shares some of its core concepts with the hierarchical JEPA models theorized in [LC22], which propose to learn a hierarchy of representations and predictors to be able to predict the evolution of a dynamical system at different time scales.

The two approaches still differ in how they leverage these representations:

- Reinforcement learning-inspired approaches learn a policy, that is to say a function which associates an optimal action to an initial state and a goal.
- JEPA-related methods, inspired by optimal control, obtain the action by optimizing a cost using an MPC procedure.

We will focus on applying the JEPA planning methods to representations learned using an offline reinforcement learning-inspired approach.

5 Improving planning with JEPAs

We wish to obtain a JEPA world model with good planning capabilities. To do so, we focus on two main ideas:

- Improving the representations used for planning.
- Leveraging a hierarchical structure.

5.1 Leveraging value functions

JEPAs learn representations using a prediction loss, with the aim of selecting the information to encode. This loss should allow the models to focus on information that is relevant for prediction, and therefore for modeling the environment. It also should encode that information in a way that makes it easy to predict future states of the environment. One way to interpret what "easy" means is that the desired representations should be such that states that are close in terms of number of time steps separating them should have representations that are close to each other.

In that case, the representation space should be such that the distance between two represented states is the cost necessary to go from one to the other. In the simplest scenario, this cost is the minimum time required to travel between the states, but it could be more complex and depend on the actions taken as well. This leads to the formulation of a new criterion for learning the representations: they should be such that the euclidean distance in the representation space is the opposite of the goal-conditioned value function of a given environment (which might be very diverse).

On top of leading to interesting representations, this criterion also has useful consequences when considering planning. Indeed, in the traditional JEPA approach, planning is done using the distance between states and a goal in the representation space as a cost. However, without any explicit condition on these distances, the resulting cost might have numerous local minima and lead to a complex optimization. However, if the representation space respects our condition, this distance is exactly the opposite of the value function and minimizing it naturally leads to the goal.

Whereas in this study we will usually consider that the representations used for prediction and for planning are the same, this is not necessary in general. Enforcing the value function criterion may lead to representations that are more biased toward the tasks present in the training dataset than those obtained when training the models with a prediction loss. To prevent this, it is possible to define two successive representation spaces. The first would correspond to a prediction loss and be used with the predictor. The other would be obtained with a second encoder and correspond to the value function criteria. It would only be used for planning.

5.1.1 Intuitive losses

To enforce the previously described criterion in the representation space, we first considered several intuitive losses, inspired by classical deep learning training approaches:

- Contrastive loss: learning the goal-conditioned value function would impose specific distances between states. However, it might be sufficient to get a representation space that only respects the order of proximity between states. States that are easy to navigate between should be close together, while those that are difficult to navigate between should be further apart. This representation space can be learned from a dataset $\mathcal{D}_{\text{c}}$ of trajectories containing positive examples $(s_t^+)_{t\in[\![0,T]\!]}$ from S_0^{T+1} and negative examples $(s_t^-)_{t\in[\![0,T-1]\!]}$ from S_0^T using a simple contrastive loss:

$$\forall((s_t^+),(s_t^-))\in\mathcal{D}_{\text{c}},\ \mathcal{L}^\theta_{\text{contrastive}}((s_t^+),(s_t^-))=\sum_{t=0}^{T-1}\|\mathcal{E}_\theta(s_{t+1}^+)-\mathcal{E}_\theta(s_t^+)\|_2^2+\sum_{t=0}^{T-1}(m-\|\mathcal{E}_\theta(s_t^-)-\mathcal{E}_\theta(s_t^+)\|_2)_+ \tag{21}$$

  , where m is a real scalar that can be tuned to control stability.

- Regressive loss: before considering the loss from IQL, one might wonder if a simpler regression loss enforcing the desired distance between representations of consecutive states in a trajectory is not

sufficient. It is possible to use the following loss with a dataset $\mathcal{D}_\text{r}$ containing trajectories $(s_t)_{t\in[\![0,T]\!]}$ from S_0^{T+1}:

$$\forall (s_t) \in \mathcal{D}_\text{r},\ \mathcal{L}^\theta_{\text{regressive1}}((s_t)) = \sum_{t=0}^{T-1} (\|\mathcal{E}_\theta(s_{t+1}) - \mathcal{E}_\theta(s_t)\|_2 - 1)^2 \tag{22}$$

Or in a similar way:

$$\forall (s_t) \in \mathcal{D}_\text{r},\ \mathcal{L}^\theta_{\text{regressive2}}((s_t) = \sum_{t=1}^{T} (\|\mathcal{E}_\theta(s_t)) - \mathcal{E}_\theta(s_0)\|_2 - t)^2 \tag{23}$$

These losses, including the traditional prediction one $\mathcal{L}_\text{pred}$, all exhibit similar properties. They enforce a proximity relationship between the representations and, without a contrastive term, they all lead to collapse. The regressive losses, for instance, impose local constraints on states that are close to each other in the trajectories, but do not control the representations of further states.

To do so, it is necessary to use a term preventing representations from all being the same, which might be the negative term of the contrastive loss. However, contrastive learning is hard in high dimension, because negative examples do not span the entirety of the space. To mitigate this issue, it is possible to use VCReg, as it is done with standard JEPA models. VCReg indeed allows to prevent any kind of collapse by inciting representations to span all of the representation space, and do not rely on explicit negative examples.

5.1.2 IQL for JEPA models

We would like to learn the parameters θ of an encoder $\mathcal{E}_\theta$ such that the euclidean distance in the associated representation space is the negative the goal-conditioned value function $V^\star$. More precisely, we wish to obtain:

$$\forall (s, g) \in \text{S}_0^2, V^\star(s, g) = -\|\mathcal{E}_\theta(s) - \mathcal{E}_\theta(g)\|_2 \tag{24}$$

In order to do so, we apply a procedure inspired by [PKL24] and that leverages the IQL loss. We use the following reaching cost: $C : s, a, g \mapsto \mathbb{1}(s \neq g)$. This cost corresponds to an environment in which every time step spent outside of the goal is penalized, and it can be applied to any reaching task, even with multiple goals of different significance.

If we have access to the actions during training, it is also possible to use a refined version of this cost: $C : s, a, g \mapsto c(a) + \mathbb{1}(s \neq g)$, with $c : \text{A}_0 \to \mathbb{R}_+$ (a norm for instance).

We define: $\forall (s, g) \in \text{S}_0^2, V_\theta(s, g) := -\|\mathcal{E}_\theta(s) - \mathcal{E}_\theta(g)\|_2$. Let $(T, N) \in \mathbb{N}^2$.

We train the encoder network with a dataset $\mathcal{D}$ of trajectories $(s_t)_{t\in[\![0,T]\!]}$ belonging to S_0^{T+1} and goals $(g_n)_{n\in[\![0,N]\!]}$ belonging to S_0^{N+1} by minimizing with gradient descent and with respect to θ the following loss:

$$\forall ((s_t), (g_n)) \in \mathcal{D},\ \mathcal{L}^\theta_{\text{VF}}((s_t), (g_n)) = \sum_{n=0}^{N} \sum_{t=0}^{T-1} L_\tau^2 \left(-\mathbb{1}(s_t \neq g_n) + \gamma V_{\bar{\theta}}(s_{t+1}, g_n) - V_\theta(s_t, g_n)\right) \tag{25}$$

, where $\bar{\theta}$ means that we use a stop gradient operator on the weights of the target network $V_{\bar{\theta}}$. Minimizing this loss across the trajectories sampled amounts to minimizing $\mathcal{L}_\text{IQL}$. In practice, we use two different types of goals:

- The last state s_T of the trajectories. They are relevant goals in the context of the trajectories and allow for a stabler training since they provide pairs (s, g) of close states.
- Random goals taken from the batch of trajectories used for training. This second type of goals notably allow introducing contrast between different trajectories. If a goal cannot be reached from a given state, the loss acts as a contrastive term that will push the two associated representations apart. We therefore hypothesized that it might be possible to replace the use of these goals by the addition of the VCReg term to the loss.

When tuning the hyperparameters of the IQL loss, we must find a compromise between convergence speed and precision. A τ close to 1 allows for a good estimate of the Bellman operator but makes convergence harder (γ_τ is then close to 1). In a similar way, a γ close to 1 allows to accurately capture relationships between distant states but leads to instability and divergence of the training (in cases where some goals cannot be reached from certain states). If we use VCReg instead of random goals, it is possible to use $\gamma = 1$ without divergence. It is also possible to set different values of γ for the different types of goals we use.

When using this approach, it is possible to only train the state encoder of the JEPA world model in a first step, and then train the predictor and the action encoder separately using the standard prediction loss $\mathcal{L}_{\text{pred}}$.

5.1.3 Relationships between losses

It is interesting to note the close relationship between the IQL-inspired loss and contrastive or regressive losses.

From a contrastive perspective, it is indeed very similar to the triplet loss [SKP15]. This loss uses a set $\mathcal{S}$ of positive states $(s_t^+)_{t\in[\![0,T]\!]}$ from S_0^{T+1}, negative states $(s_t^-)_{t\in[\![0,T]\!]}$ from S_0^{T+1} and anchor states $(g_t)_{t\in[\![0,T]\!]}$ from S_0^{T+1}:

$$\forall((s_t^+),(s_t^-),(g_t)))\in\mathcal{S},\ \mathcal{L}^\theta_{\text{triplet}}((s_t^+),(s_t^-),(g_t))=\sum_{t=0}^{T}(\|s_t^+-g_t\|_2^2-\|s_t^--g_t\|_2^2+m)_+ \tag{26}$$

, where m is a margin to control stability. If we consider that positive states are states from a trajectory, negative states their successive states, anchor states the goals, that the margin is negative, and do not use the stop gradient, the $\mathcal{L}_{\text{VF}}$ loss is very similar to this loss for $\tau = 1$.

From a regressive perspective, if we always define the goal as the state directly following of a given state, and do not use the stop gradient, the $\mathcal{L}_{\text{VF}}$ loss becomes:

$$\forall((s_t),(g_n))\in\mathcal{D},\ \mathcal{L}^\theta_{\text{VF}}((s_t))=\sum_{t=0}^{T-1}L_\tau^2\left(-\mathbb{1}(s_t\neq s_{t+1})-V_\theta(s_t,s_{t+1})\right) \tag{27}$$

which is the same as $\mathcal{L}_{\text{regressive1}}$ for $\tau = 0.5$ (up to a scalar factor).

5.1.4 Properties of the representations

Let us suppose that we have learned representations that satisfy our value function criterion for the type of reaching costs considered. Interesting properties arise. We suppose here that all networks are perfectly trained: the distance in the representation space is exactly the negative goal-conditioned value function, and the predictor always outputs the correct result.

To begin with, such representations make the optimization of the action planning problem more robust. Indeed, let us consider a problem where one aims to reach a goal g in S_0. For simplicity, we suppose that this goal can be reached from any state in S_0. When using distance in an arbitrary representation space, in general, we have:

$$\exists s\in\mathrm{S}_1,\forall a\in\mathrm{A}_1,\|\mathcal{P}(s,a)-\mathcal{E}(g)\|_2\geqslant\|s-\mathcal{E}(g)\|_2 \tag{28}$$

The planning process is therefore prone to get stuck in this kind of local minima when optimizing an action.

However, this is not true if the representations respect the value function criteria. In that case:

$$\forall s\in\mathrm{S}_1,\exists a\in\mathrm{A}_1,\exists t\in\mathbb{N},\|\mathcal{P}(s,a)-\mathcal{E}(g)\|_2=\|s-\mathcal{E}(g)\|_2-\gamma^t<\|s-\mathcal{E}(g)\|_2 \tag{29}$$

This inequality holds even if the function learned corresponds to a noisy goal-conditioned value function, as long as the noise intensity is strictly smaller than γ^t. With such representations, optimizing actions greedily is therefore sufficient to reach the goal, as long as it is reachable from the starting state.

Moreover, if the goal-conditioned value function is perfectly learned, then the representations of the states of the optimal trajectory that goes from an observed state o in S_0 to a goal g in S_0 will be approximately aligned. Indeed, let $s \in \mathrm{S}_0$ be the t^{th} point of the trajectory. Since it is optimal, we have:

$$V^\star(o, g) = V^\star(o, s) + \gamma^n V^\star(s, g) \tag{30}$$

and therefore

$$\|\mathcal{E}(o) - \mathcal{E}(g)\|_2 = \|\mathcal{E}(o) - \mathcal{E}(s)\|_2 + \gamma^t \|\mathcal{E}(s) - \mathcal{E}(g)\|_2 \tag{31}$$

which means that $\mathcal{E}(o)$, $\mathcal{E}(s)$ and $\mathcal{E}(g)$ are approximately aligned if $\gamma \approx 1$.

This should be put into perspective by the fact that it is not possible, in general, to obtain representations that exactly respect this condition, and $V^\star$ will only be approximated.

5.1.5 Potential issues

The function learned using our procedure is likely to be quite different from the real goal-conditioned value function in practice.

a) Different models for $V^\star$

First of all, its ability to model the goal-conditioned value function is limited by the fact that it is based on a distance. The opposite of the goal-conditioned value function $V^\star$ with goal-reaching cost:

- Is positive: $\forall (s, g) \in \mathrm{S}_0^2, -V^\star(s, g) \geqslant 0$
- Respects the identity of indiscernibles: $\forall (s, g) \in \mathrm{S}_0^2, s = g \Leftrightarrow V^\star(s, g) = 0$
- Respects the triangle inequality: $\forall (s, s', g) \in \mathrm{S}_0^2, -V^\star(s, g) \leqslant -V^\star(s, s') - V^\star(s', g)$
- Is not symmetric in general. There exist an environment in which: $\exists (s, g) \in \mathrm{S}_0^2, V^\star(s, g) \neq V^\star(g, s)$. For instance, an agent could be able to go down a slope but not to climb it.

The fact that $V^\star$ is positive and respects the identity of indiscernibles and the triangle inequality justifies basing its approximation on a distance. However, it cannot be learned exactly with our current approach since a distance is symmetric. To try to obtain a better approximation, it is possible to replace the euclidean distance by a quasimetric distance d_{quasi} on S_1. The goal-conditioned value function indeed respects all of the axioms of a quasimetric. We would then learn $V^\star$ with a function V_θ such that:

$$\forall (s, g) \in \mathrm{S}_0^2, V_\theta(s, g) = -d_{\text{quasi}}(\mathcal{E}_\theta(s), \mathcal{E}_\theta(g)) \tag{32}$$

This idea is explored in [Wan+23], which uses for d_{quasi} a family of quasidistances developed in [WI22]: interval quasimetric embeddings (IQE).

Let us define what an IQE distance is. Let $(d_1, d_2) \in \mathbb{N}^2$. Let $(s, s') \in (\mathbb{R}^{d_1 d_2})^2$ be two representation vectors, that we reshape into matrices $(s_{i,j})_{i \in [\![1, d_1]\!], j \in [\![1, d_2]\!]}$ and $(s'_{i,j})_{i \in [\![1, d_1]\!], j \in [\![1, d_2]\!]}$. We define:

$$\forall i \in [\![1, d_1]\!], d_i(s, s') = \left| \bigcup_{j=1}^{d_2} [s_{i,j}, \max(s_{i,j}, s'_{i,j})] \right| \tag{33}$$

where $|\cdot|$ is the Lebesgue measure.

In our experiments, we will use the maxmean IQE distance, which is defined as:

$$d_{\text{IQE}}(s, s') = \alpha \max_{i \in [\![1, d_1]\!]} (d_i(s, s')) + (1 - \alpha) \frac{1}{d_1} \sum_{i=1}^{d_1} d_i(s, s') \tag{34}$$

where α belongs to $[0, 1]$ and is a learnable parameter.

The authors of [WI22] prove that when used in a representation space, the combination of the encoder and the IQE distance can approximate any quasidistance. They also observe good empirical performance on tasks that require learning a quasidistance.

It is also possible to replace the distance by an unconstrained deep learning network, but removing all the restrictions makes it hard for the training to converge without collapsing.

b) Influence of the dataset

One can also wonder about the choice of the policy π used to create the training dataset. The theoretical results about the IQL loss show that only the support of π actually matters when τ tends to 1 (the Bellman equation depends solely on this support). Therefore, our approach only requires a policy that has a nonzero probability of using every action available.

In practice, however, it is likely that the support of π is not the only parameter that matters. In very suboptimal trajectories, states that should be close to each other appear far apart and make the optimization harder. Therefore, it might be preferable to use "expert" trajectories of good quality. However, this often comes at the price of diversity and exploration, limiting the capacities of the resulting model.

The extent to which trajectories span the space has an impact on the representations learned. It is important that the states used in the IQL loss during training span the entire space of observations. In the case of a random environment, the whole observation space should be covered for every configuration of the environment. The generalization capabilities of the model are so far unclear from a theoretical point of view.

c) Locality of the training

Finally, while local relationships between states can be expected to be correctly encoded, this is more unlikely for distant relationships, for two main reasons:

- While the density of close states (like (s_t, s_{t+1}, s_T)) used for training is high, the distant states seen during training, through the random goals or VCReg, sparsely occupy the space they belong to.
- We mentioned that when representations respect the value function criteria, it is possible to optimize actions greedily to plan, even if the goal-conditioned value function is not perfectly learned. This is true as long as the noise intensity is strictly smaller than γ^t, where t is the minimal number of actions required to reach the goal from the state considered. However, if the starting state s from S_0 is far from the goal, t is large, and $\gamma^t \approx 0$. Therefore, if the learned value function is affected by a noise of constant intensity, which is typically the case, the property is not true for distant states and planning can get stuck in local minima.

This suggests that using a hierarchy of representation spaces, trained with an increasing number of actions between triplets of states, might allow to capture more distant relationships and yield better results.
This is also the case when using the standard prediction loss to train the world model.

5.2 Leveraging hierarchical models

We are interested in obtaining a hierarchical world model, which could leverage different levels of abstraction to plan over longer horizons and tackle tasks of different levels of complexity. We distinguish between two types of hierarchies: one of states and one of actions. A hierarchy of states would allow for better capturing relationships between distant states. The one of actions can increase the planning horizon by reducing prediction errors and computational costs at larger scales.

5.2.1 Training a hierarchy

We aim to train additional representation levels that focus on different scales of relationships between states of an environment.

To do so, we define a hierarchy of state spaces $(\mathrm{S}_\ell)_{\ell\in[\![0,N]\!]}$, of action spaces $(\mathrm{A}_\ell)_{\ell\in[\![0,N]\!]}$, and a sequence of subsampling strides $(k_\ell)_{\ell\in[\![1,N]\!]}$. We aim to learn the following networks:

- State encoders :$(\mathcal{E}^\ell : \mathrm{S}_{\ell-1} \to \mathrm{S}_\ell)_{\ell\in[\![1,N]\!]}$, which allow obtaining the hierarchical state representations.
- Action encoders: $(\mathcal{A}^\ell : \mathrm{A}_{\ell-1}^{k_\ell} \to \mathrm{A}_\ell)_{\ell\in[\![1,N]\!]}$, that allow obtaining the hierarchical action representations. They take as input a sequence of successive actions.
- Predictors: $(\mathcal{P}^\ell : \mathrm{S}_\ell \times \mathrm{A}_\ell \to \mathrm{S}_\ell)_{\ell\in[\![1,N]\!]}$, which compute predictions at a given level.

We define a hierarchical world model as: $(\mathcal{W}^\ell)_{\ell\in[\![1,N]\!]} = (\mathcal{E}^\ell, \mathcal{A}^\ell, \mathcal{P}^\ell, k_\ell)_{\ell\in[\![1,N]\!]}$

We train all the networks with the same losses as before (namely $\mathcal{L}_{\text{VCReg}}$, $\mathcal{L}_{\text{VF}}$, $\mathcal{L}_{\text{contrastive}}$, etc...), but with different sequences of states and actions, which we define inductively. They are obtained by successive subsamplings of the trajectories of the initial training dataset using the strides $(k_\ell)_{\ell\in[\![1,N]\!]}$.

Let $T_0 \in \mathbb{N}$. We originally have access to a dataset $\mathcal{D}^0$ of trajectories $((s_t^0)_{t\in[\![0,T_0]\!]}, (a_t^0)_{t\in[\![0,T_0-1]\!]})$ belonging to $\mathrm{S}_0^{T_0+1} \times \mathrm{A}_0^{T_0}$. We start by training $\mathcal{E}^1$, $\mathcal{A}^1$ and $\mathcal{P}^1$ using a subsampling stride k_1 (in practice, $k_1 = 1$) and the same losses as with a standard world model.

For a given level $\ell \in [\![1, N-1]\!]$, let us suppose that we have trained the models $\mathcal{E}^\ell$, $\mathcal{A}^\ell$ and $\mathcal{P}^\ell$, and that we have access to the dataset $\mathcal{D}^{\ell-1}$ of trajectories used for their training. We define $T_\ell = \lfloor T_{\ell-1}/k_\ell \rfloor$. We compute for every trajectory $((s_t^{\ell-1})_{t\in[\![0,T_{\ell-1}]\!]}), (a_t^{\ell-1})_{t\in[\![0,T_{\ell-1}-1]\!]}))$ of $\mathcal{D}^{\ell-1}$:

$$\forall t \in [\![0, T_\ell]\!], s_t^\ell = \mathcal{E}^\ell(s_{k_\ell t}^{\ell-1}) \tag{35}$$

and

$$\forall t \in [\![0, T_\ell - 1]\!], a_t^\ell = \mathcal{A}^\ell\left((a_j^{\ell-1})_{j\in[\![k_\ell t, k_\ell(t+1)-1]\!]}\right) \tag{36}$$

We then use the dataset $\mathcal{D}^\ell$ composed of the $((s_t^\ell)_{t\in[\![0,T_\ell]\!]}, (a_t^\ell)_{t\in[\![0,T_\ell-1]\!]})$ to train $\mathcal{E}^{\ell+1}$, $\mathcal{A}^{\ell+1}$ and $\mathcal{P}^{\ell+1}$ with the usual procedure (with subsampling stride $k_{\ell+1}$).

The procedure applied to obtain the hierarchy of training trajectories can be visualized on Fig. 4.

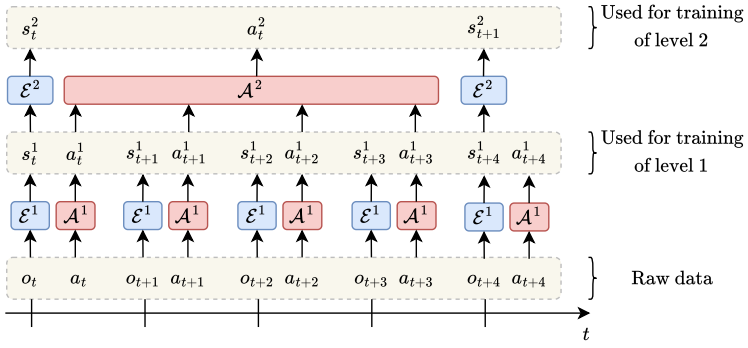


Figure 4: Diagram of the data used to train a hierarchical model

5.2.2 Planning with a hierarchy

Now that we have access to trained hierarchies of actions and states, we can perform planning.

a) Leveraging the state hierarchy

Once we have trained the hierarchy of state encoders, we can use them for action planning to go from a start state defined by an observation o from S_0 to a goal g in S_0. We can do so in a way that does not

uses the action hierarchy, and only requires the first predictor $\mathcal{P}^1$ and action encoder $\mathcal{A}^1$. We change the cost used in the usual planning procedure optimizing a sequence of actions $(a_t)_{t\in[\![0,T-1]\!]}$ with horizon T to include all representation levels. The optimization problem becomes:

$$\begin{aligned} \underset{(a_t)_{t\in[\![0,T-1]\!]}\in \mathrm{A}_1^T}{\text{minimize}} \quad & \mathcal{L}_{\text{plan}} = \sum_{\ell=1}^{N}\sum_{t=1}^{T} \|s_t^\ell - s_g^\ell\|_2 \\ \text{such that} \quad & \begin{cases} s_0^1 = \mathcal{E}^1(o) \\ \forall t \in [\![0, T-1]\!],\ s_{t+1}^1 = \mathcal{P}^1(s_t^1, a_t) \\ \forall \ell \in [\![2, N]\!], \forall t \in [\![1, T]\!], s_t^\ell = \mathcal{E}^\ell \circ \mathcal{E}^{\ell-1} \circ \cdots \circ \mathcal{E}^2(s_t^1) \\ \forall \ell \in [\![1, N]\!], s_g^\ell = \mathcal{E}^\ell \circ \mathcal{E}^{\ell-1} \circ \cdots \circ \mathcal{E}^1(g) \end{cases} \end{aligned} \tag{37}$$

This cost should provide access to relevant information at all scales during planning.

However, this approach is still limited. It would be interesting to leverage the hierarchy better and to perform action planning inside the different levels of representation.

b) Leveraging the action hierarchy

To perform action planning with the hierarchy of actions, we take inspiration from the MPC procedure. The key step of MPC is the optimization of a sequence of actions which, when applied from a starting state, allow one to get closer to a goal. We propose a method that outputs these actions by leveraging the different representation levels. The idea is to use planning in higher levels to get intermediate targets for the lower levels.

To perform this planning step, we adopt an inductive approach.
We define: $\forall \ell \in [\![1, N]\!], \forall x \in \mathrm{S}_0, \mathcal{E}_\ell(x) = \mathcal{E}^\ell \circ \mathcal{E}^{\ell-1} \circ \cdots \circ \mathcal{E}^1(x)$. We also fix the planning horizons: $(T_\ell)_{\ell\in[\![1,N]\!]} \in \mathbb{N}^{N+1}$.

We first compute the highest representations of the starting state o in S_0 and the goal state g in S_0, namely $\mathcal{E}_N(o)$ and $\mathcal{E}_N(g)$. We then apply a standard planning procedure (leveraging MPPI, CEM, etc...) in S_N and A_N with horizon T_N. Doing so, we obtain a sequence of states $(s_t^N)_{t\in[\![0,T_N]\!]}$ belonging to $\mathrm{S}_N^{T_N+1}$ such that $s_0^N = \mathcal{E}_N(o)$, and the corresponding sequence of actions $(a_t^N)_{t\in[\![0,T_N-1]\!]}$ belonging to $\mathrm{A}_N^{T_N}$.

For a given ℓ in $[\![1, N-1]\!]$, let us suppose that we have access to the previous sequence of states $(s_t^{\ell+1})_{t\in[\![0,T_{\ell+1}]\!]}$ belonging to $\mathrm{S}_{\ell+1}^{T_{\ell+1}+1}$. We use the second state of this sequence $s_1^{\ell+1}$ as an intermediate goal for planning in the lower layer.

We perform action optimization in S_ℓ and A_ℓ using a cost similar to the one of the standard procedure, $\mathcal{E}_\ell(o)$ as starting point, $s_1^{\ell+1}$ as goal and T_ℓ as horizon. Namely, we solve:

$$\begin{aligned} \underset{(a_t)_{t\in[\![0,T_\ell-1]\!]}\in \mathrm{A}_\ell^{T_\ell}}{\text{minimize}} \quad & \mathcal{L}_{\text{plan}} = \sum_{t=1}^{T_\ell} \|\mathcal{E}^{\ell+1}(s_t) - s_1^{\ell+1}\|_2 \\ \text{subject to} \quad & \begin{cases} s_0 = \mathcal{E}_\ell(o) \\ \forall t \in [\![1, T_\ell - 1]\!],\ s_{t+1} = \mathcal{P}^\ell(s_t, a_t) \end{cases} \end{aligned} \tag{38}$$

For $\ell = 1$, this procedure allows to obtain the sequence of actions to be applied. If $\mathcal{A}^1 \neq \mathrm{Id}$, this sequence needs to be decoded in order to obtain actions that can be applied in the real environment. In practice, we use $\mathcal{A}^1 = \mathrm{Id}$.

This planning process can be visualized in Fig. 5.

It is possible to adapt what levels of the hierarchy we use depending on the distance between the current state and the goal in the representation spaces. If this distance is lower than a given threshold in the highest space, it is possible to start the previous procedure in a lower space.

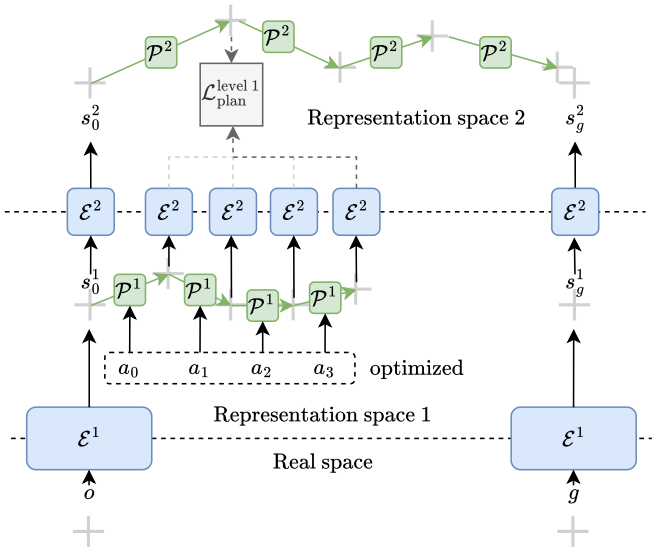


Figure 5: Planning in an intermediate level of a hierarchical model - computing the cost in the higher level

c) Adversarial examples

An important issue arises when planning with the hierarchical procedure described. When optimizing abstract actions, there are no guarantees a priori that the output actions will be realistic, namely that they will correspond to an encoding of real actions. On the contrary, it is likely that they will constitute an adversarial example [GSS15] optimizing the output of the predictor in an unwanted way. This effect is less significant when we plan in the lowest level with an action encoder equal to the identity since the action space is of small dimension and usually dense.

To solve this issue, our idea is to take inspiration from variational auto-encoders (VAEs) [KW22]. A VAE is composed of two networks: an encoder and a decoder. Its goal is to learn the distribution underlying a training dataset in order to be able to sample new data following it. To do so, the encoder takes as input a training sample, and outputs the mean and standard deviation of the distribution q of its encodings, that is supposed gaussian. A random vector is sampled according to this distribution, and input in the decoder.

The network is trained using a reconstruction loss (such as the mean square error for instance) between the output of the decoder and the initial sample, as well as the Kullback-Leibler (KL) divergence between q and the normal distribution. The space between the encoder and the decoder is usually called the latent space, and is equal to $\mathbb{R}^n$ for a chosen integer n.

A key property of VAEs is that their decoder can theoretically map any vector from the latent space to a realistic data sample. A theoretically perfect decoder indeed transforms the prior distribution of the latent space into the training data distribution. An intuitive way to understand this is that the decoder is trained with vectors sampled following a distribution that theoretically span the whole latent space. In practice however, the decoder is not perfect and is most accurate in regions of latent space with high prior probability, since they are the regions most frequently encountered during training.

Let us call $\mathcal{R}_a$ the set of representations of real actions at a given level of the hierarchy. In our settings, VAEs could provide a way to learn a network that approximately maps $\mathbb{R}^n$ to $\mathcal{R}_a$, and to ensure that we always plan with realistic actions.

A way to adapt the VAE approach to our architecture is to consider that the action encoder is the encoder and that the predictor contains the decoder of a VAE. In that case, we only have to make the following changes during training:

- Make the action encoder output the mean μ and standard deviation σ of the encodings. Use as input in the predictor a vector sampled with a gaussian distribution of mean μ and standard deviation σ.
- Add a KL regularization term $\mathcal{L}_{\text{KL}}$ to the loss, that is applied on the action encodings:

$$\forall(\mu,\sigma) \in \mathbb{R}^n \times \mathbb{R}_+^n,\ \mathcal{L}_{\text{KL}}(\mu,\sigma) = \frac{1}{2}(-\log(\sigma^2) - 1 + \sigma^2 + \mu^2) \tag{39}$$

  where n is the dimension of the action representation space. We supposed that the prior on the action representation space is the normal distribution.

Overall, we therefore keep a similar approach as for a VAE but replace the reconstruction loss by a prediction loss. When planning, to make sure that the actions output by the procedure belong to regions of the space with high prior probability, we also add a penalization on the norm of actions in the planning cost.

One can observe that the planning procedure we described does not use the first representation level explicitly for the cost. This is because we chose to step down in the hierarchy through action optimization. An alternative possibility would be, at a given level ℓ, to do so by optimizing an intermediate goal state s_g^ℓ such that: $\mathcal{E}^{\ell+1}(s_g^\ell) = s_1^{\ell+1}$ (as before, $s_1^{\ell+1}$ is the second state of the trajectory planned in the higher level). This goal state could then be used for planning in the lower level with the standard procedure, as visualized on Fig. 6. However, by doing so, we would be faced with the adversarial attack issue when optimizing the state. It would therefore be necessary to regularize the states as well.

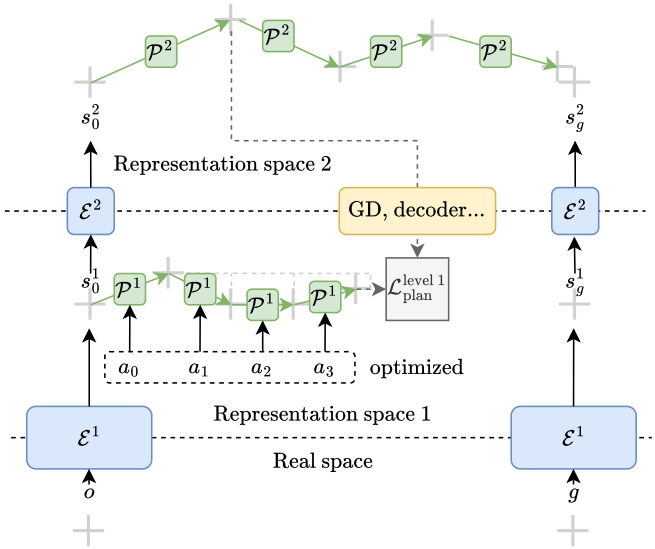


Figure 6: Planning in an intermediate level of a hierarchical model - optimization of the intermediate goal

Another way to tackle this planning optimization issue would be to train decoders to transform the representations of a high level of the hierarchy into those of a lower level. It is also possible to consider an action hierarchy without a state hierarchy. In such a case, all planned trajectories would belong to the same space.

6 Results

6.1 First experiments

Before applying our value function learning approach to complex environments, we test it on very simple ones to assess its initial performances. They consist of an agent navigating in a room with walls that it cannot cross. The action space is continuous and consists of 4 logits. The action performed by the agent is sampled randomly using these logits and corresponds to a movement up, down, left or right. We train a simple convolutional state encoder $\mathcal{E}$ using $\mathcal{L}_{\text{VF}}$ (with $\gamma = 0.95$ and $\tau = 0.7$).

To visualize the value function for a given goal in these experiments, we will plot the distance in the representation space between any possible position of the agent and the goal. We expect it to be similar to the optimal value function in our environments, that is to say the minimum number of actions required to reach the goal from a given state.

6.1.1 Value function

To assess to what extent the distance in the representation space can approximate the goal-conditioned value function, we use an environment of size 6×6 with the configuration displayed on Fig. 7. It is interesting because the goal-conditioned value function in this environment is very different from the euclidean distance between the positions of the agent. We use 100000 trajectories of length 32 for training.

We consider two states (s_0, s_1) from S_0^2 of coordinates $(1, 4)$ and $(6, 1)$:

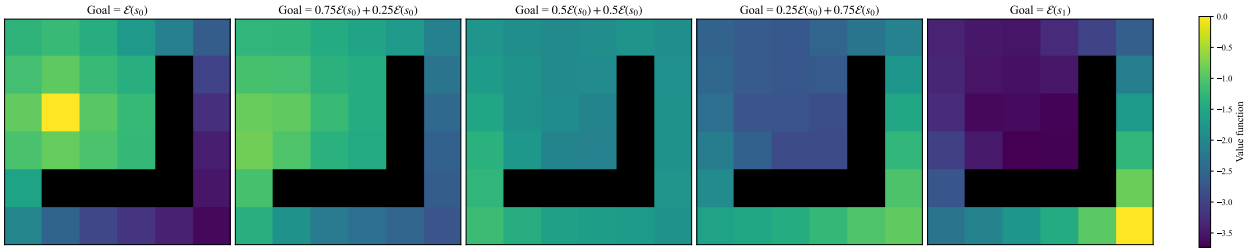


Figure 7: Value function visualization for different goals (obstacle in black)

We observe that we obtain reasonable value functions. Moreover, it appears that the representations of optimal trajectories tend to be aligned in the representation space. If we use as goals states interpolated between the representations of s_0 and s_1, the states maximizing the value function correspond to the shortest path between these s_0 and s_1.

6.1.2 Generalization

To assess the generalization capabilities of our approach, we use an environment of size 6×6 with random obstacles, such as the ones displayed on the two leftmost examples of Fig. 9. We use 100000 trajectories of length 32 for training.

We tested it on new random configurations:

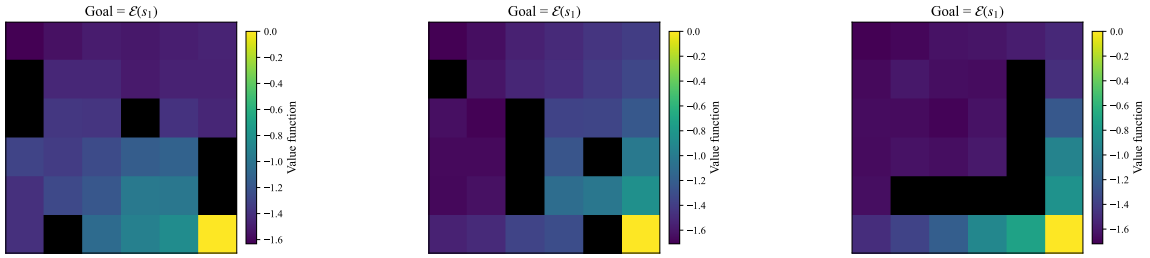
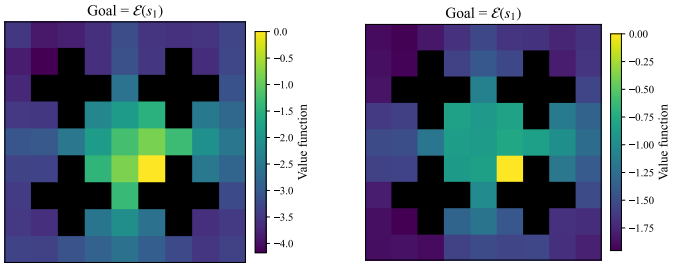


Figure 8: Value function visualization for different configurations of the environment

These results confirm the generalization capabilities of our goal-conditioned value function approach.

6.1.3 Hierarchy

We evaluate the effect of learning a goal-conditioned value function with subsampled trajectories. To do so, we train an encoder using two different types of trajectories of length 4: the first contains states separated by a single action, and the other states separated by 10 actions. The first type of trajectory corresponds to what would be the first level of a hierarchy of representations, and the other to a higher level. We use 10000 trajectories for training.



(a) Lower level (b) Higher level

Figure 9: Value function visualization for different subsampling rates (goal s_1 of coordinates $(6, 4)$)

We observe that local relationships between points are well encoded in the lower level, but the ones between distant points are quite inaccurate. In the higher level, the representations of close points tend to be merged together, but those of distant points are better encoded. A new structure, in which every 'room' of the environment corresponds to a single state, starts to appear. This seems desirable and confirms that using subsampled trajectories allows learning representations capturing different levels of abstraction of the environment.

6.2 Test environments

We will realize our experiments with different environments and with an offline reinforcement learning approach. For all of them, the states used as inputs of our models are observation images of size 64×64, potentially with proprioceptive information.

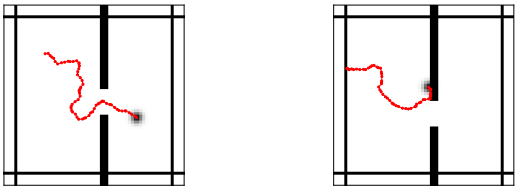
6.2.1 Wall

The wall environment consists of a square space separated in two by a wall with a door. The agent, a point, has to go from a starting point on one side of the wall to a target point on the other side. To do so, it can execute actions that are vectors corresponding to a displacement, and it has to pass through the door. The observations of states of this environment have 2 channels, one corresponding to the agent, the other to the walls. A visualization of a typical state of this environment (with flattened channels) can be found on Fig. 10 and 11.

To generate the dataset of training trajectories, we take inspiration from [Sob+25], and do not simply sample actions with a Gaussian noise. This would indeed lead to trajectories that remain concentrated in a small region of space. Instead, we generate actions by sampling a random direction, which we then perturb with noise drawn from the von Mises distribution (a circular version of the normal distribution) with concentration parameter 5. We generate datasets containing 1000 trajectories of length 64.

We will generate datasets with different settings for this environment:

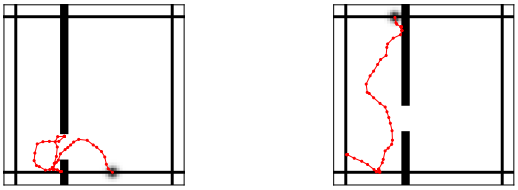
- Wall_small, corresponding to a wall and a door with random positions across trajectories, and actions sampled randomly with mean 1 pixel and standard deviation 0.4, and clipped to the range $[0.2, 1.8]$.



(a) Crossing trajectory (b) Non crossing trajectory

Figure 10: Examples of trajectories from the Wall_small dataset

- Wall_big, corresponding to a wall and a door with random positions across trajectories, and actions sampled randomly with mean 2 pixels and standard deviation 0.8, and clipped to the range $[0.4, 3.6]$.



(a) Crossing trajectory (b) Non crossing trajectory

Figure 11: Examples of trajectories from the Wall_big dataset

When generating the datasets, we make them such that half the trajectories they contain correspond to the agent passing through the door.

6.2.2 Maze

The maze environment setting is the one used in [Sob+25], which is based on the Mujoco PointMaze environment [Fu+21]. It is based on a grid of 4×4 squares. Among them, between 50% and 60% contiguous ones are selected to create the maze. The agent has to move from a random starting square to a random goal square. The observations of states of this environment are colored and have 3 channels. An example of the environment can be visualized on Fig. 12.

To assess the generalization capabilities of the different approaches we experiment with, we adopt the same approach as in [Sob+25]. The training trajectories all belong to five maze layouts, that are different from the ones used to evaluate planning. The dataset contains 1000 trajectories of length 101.

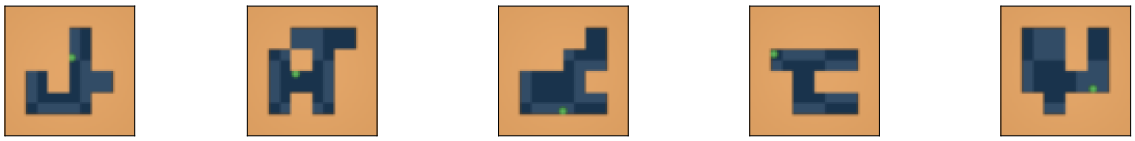


Figure 12: The 5 training layouts (the agent is the green point)

A key difference between this environment and the wall one is that the agent is a point mass, which means that it is parametrized both by its location and its speed. Its actions are target speeds to reach. The environment then computes the force that should be applied on the agent to reach the desired speed after a certain number of time steps. The trajectories are generated by sampling random speed vectors with norms smaller than 5, starting from a random position. To evaluate the planning capabilities of our approaches

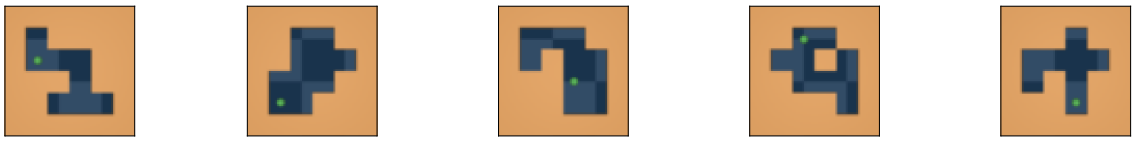


Figure 13: Examples of test layouts (the agent is the green point)

in this environment, random starting points and goals are sampled, such that they are at least 3 cells away from each other.

To plan in this environment, it is necessary that both the location and the velocity of the agent are encoded in the representations. In a similar approach to [Sob+25], and since our encoders only deal with single images, we add as input to the encoders "proprioceptive" information, namely the velocity of the agent in a given state.

6.3 Impact of the representations

We conducted tests to assess the performance of different learning approaches. Namely, we consider learning representations by training:

- pred_VCReg: all models at the same time with $\mathcal{L}_{\text{pred}}$ and $\mathcal{L}_{\text{VCReg}}$, as with standard JEPA world models [Sob+25].
- pred_EMA: all models at the same time with $\mathcal{L}_{\text{pred}}$ and an EMA procedure, as with standard JEPA representation models [Zho+25].
- Contrastive: first the state encoder with $\mathcal{L}_{\text{contrastive}}$, and then the predictor and action encoder with $\mathcal{L}_{\text{pred}}$.
- Regressive1: first the state encoder with $\mathcal{L}_{\text{regressive1}}$ and $\mathcal{L}_{\text{VCReg}}$, and then the predictor and action encoder with $\mathcal{L}_{\text{pred}}$.
- Regressive2: first the state encoder with $\mathcal{L}_{\text{regressive2}}$ and $\mathcal{L}_{\text{VCReg}}$, and then the predictor and action encoder with $\mathcal{L}_{\text{pred}}$.
- VF_VCReg: first the state encoder with $\mathcal{L}_{\text{VF}}$ without random goals and $\mathcal{L}_{\text{VCReg}}$, and then the predictor and action encoder with $\mathcal{L}_{\text{pred}}$.
- VF: first the state encoder with $\mathcal{L}_{\text{VF}}$ with random goals, and then the predictor and action encoder with $\mathcal{L}_{\text{pred}}$.
- VF_quasi: first the state encoder with $\mathcal{L}_{\text{VF}}$ with random goals and a value function using a quasimetric, and then the predictor and action encoder with $\mathcal{L}_{\text{pred}}$.
- VF_quasi_pred: all models at the same time with $\mathcal{L}_{\text{VF}}$ with random goals, a value function using a quasimetric and with $\mathcal{L}_{\text{pred}}$.
- VF_VCReg: all models at the same time with $\mathcal{L}_{\text{VF}}$ without random goals, $\mathcal{L}_{\text{VCReg}}$, and $\mathcal{L}_{\text{pred}}$.
- VF_pred: all models at the same time with $\mathcal{L}_{\text{VF}}$ with random goals and $\mathcal{L}_{\text{pred}}$.

By default, we will use flat representations of size 512, a predictor with a MLP architecture and an action encoder equal to the identity. The encoder is based on a simple architecture leveraging convolutions and residual connections. Before the predictor, the representations of the states and actions are concatenated. The encoder has 2.2M parameters and the predictor has 1.3M parameters. More information about the architectures used can be found in the Appendix 8.3.

6.3.1 On the wall environment

We will assess the quality of the representations learned on the wall environment using two methods:

- A qualitative one: a visualization of the value function learned by the model, just as for the initial simple environments. It is representative of the cost used for planning and provides good insights into its landscape.
- A quantitative one: the planning accuracy of the model, that is to say the proportion of successful plans for random pairs of starting states and goals. We compute it with 200 environments so that the variance of the results is small.

All networks were trained with a base learning rate of 0.0028, an Adam optimizer and a cosine schedule. The VCReg loss is only computed along the batch dimension of the representations.

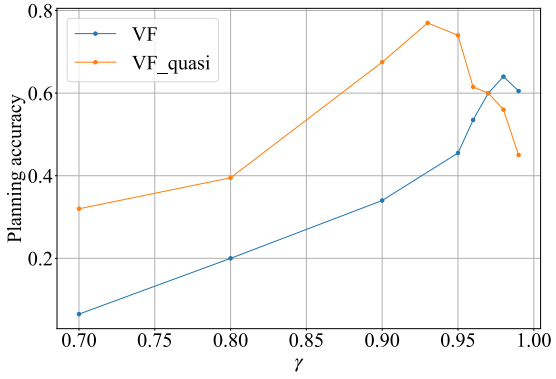
We will compare the planning results obtained when using the MPPI, CEM and stochastic gradient descent optimization methods. We use the following parameters:

- 2000 initial perturbations are sampled from a Gaussian distribution with mean 0 and standard deviation 12, using a temperature parameter of $\lambda = 0.005$.
- CEM: 5 optimization steps. For each step, the top 10 action sequences are selected out of 500. The initial actions are sampled from a Gaussian distribution with mean 0 and standard deviation 4.
- SGD: 30 steps with a learning rate of 0.3.

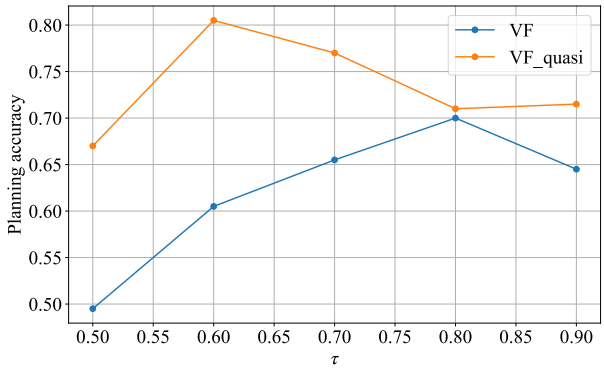
We use a planning horizon of 96, for a total of 200 planning steps for the Wall_small environment. We use a planning horizon of 64, for a total of 64 planning steps for the Wall_big environment. These settings will be used for all following experiments.

a) Hyperparameters for value function learning

To begin with, we optimized the two main hyperparameters controlling the behavior of the value function learning methods, namely τ and γ. We did so on a Wall_small environment dataset that is different than the one we will use for the rest of the tests. Here are our planning results:



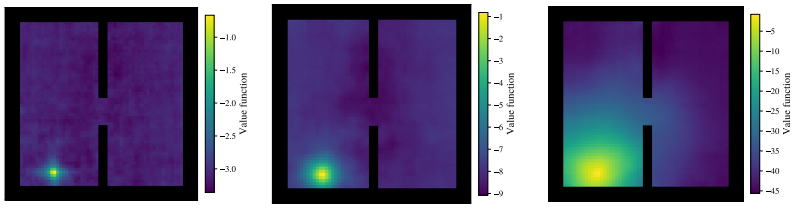
(a) Results for γ ($\tau = 0.7$)



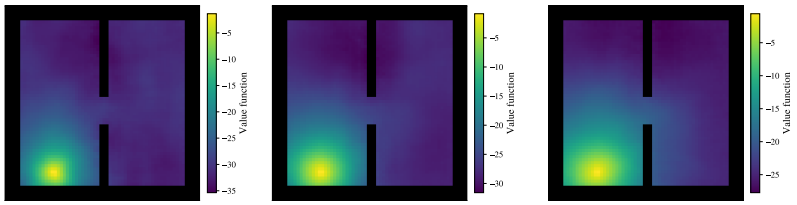
(b) Results for τ ($\gamma = 0.98$ for VF, $\gamma = 0.93$ for VF_quasi)

Figure 14: Evolution of the planning accuracy with hyper parameters

We also display visualizations of the value functions:

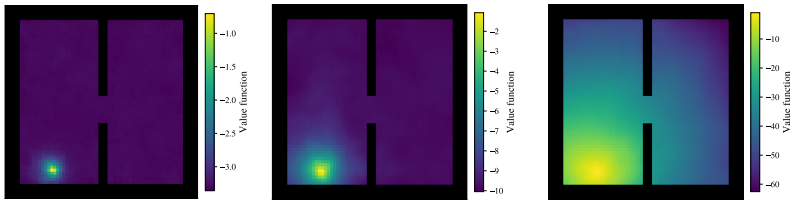


(a) $\gamma = 0.7, \tau = 0.7$ (b) $\gamma = 0.9, \tau = 0.7$ (c) $\gamma = 0.99, \tau = 0.7$

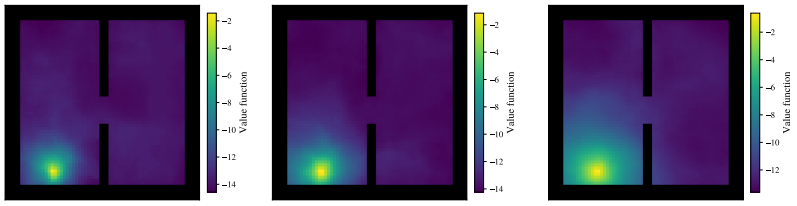


(d) $\gamma = 0.98, \tau = 0.5$ (e) $\gamma = 0.98, \tau = 0.7$ (f) $\gamma = 0.98, \tau = 0.9$

Figure 15: Visualization of the value function for the VF approach and a given goal



(a) $\gamma = 0.7, \tau = 0.7$ (b) $\gamma = 0.9, \tau = 0.7$ (c) $\gamma = 0.99, \tau = 0.7$



(d) $\gamma = 0.93, \tau = 0.5$ (e) $\gamma = 0.93, \tau = 0.7$ (f) $\gamma = 0.93, \tau = 0.9$

Figure 16: Visualization of the value function for the VF approach and a given goal

We observe that γ has a direct effect on the spread of the value function. Increasing it improves performance, as it better captures the relationships between distant points. However, setting it too close to 1 introduces instabilities that result in poorer performance.

The parameter τ is linked to the maximum operator in the Bellman equation. Its effect on accuracy follows a trend similar to that of γ. Theoretically, it should be set as close to 1 as possible, but values that are too high can lead to unstable training and poorer performance.

For the rest of the experiments, we will use $\gamma = 0.98$ and $\tau = 0.8$ for the VF-based approaches ; and $\gamma = 0.93$ and $\tau = 0.6$ for the VF_quasi-based ones.

b) Planning results

Using the settings previously described, we obtained the following results:

Type	MPPI	CEM	SGD
Contrastive	0.485	0.420	0.420
Regressive1	0.540	0.450	0.560
Regressive2	0.605	0.470	0.580
pred_VCReg	0.550	0.310	0.460
pred_EMA	0.455	0.435	0.200
VF	0.630	**0.515**	**0.655**
VF_pred	0.550	0.465	0.530
VF_quasi	**0.710**	0.470	0.635
VF_quasi_pred	0.605	0.480	0.495
VF_VCReg	0.485	0.320	0.545
VF_VCReg_pred	0.470	0.270	0.440

Table 1: Planning results for Wall_small

Type	MPPI
Contrastive	0.585
Regressive1	0.565
Regressive2	0.575
pred_VCReg	0.890
pred_EMA	0.430
VF	0.935
VF_pred	0.750
VF_quasi	**0.955**
VF_quasi_pred	0.850
VF_VCReg	0.745
VF_VCReg_pred	0.670

Table 2: Planning results for Wall_big

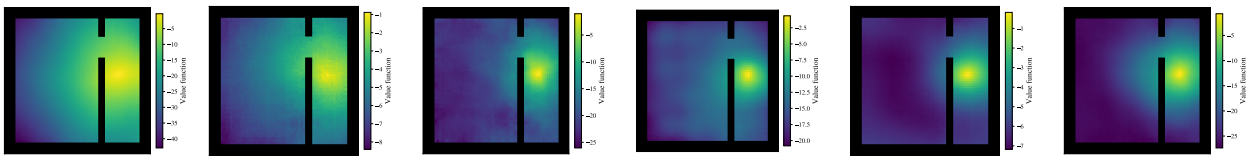
Therefore, IQL-inspired approaches seem to provide valuable information during planning and lead to results that are better than those of intuitive and prediction-based approaches. It is interesting to observe that the VF_quasi approach outperforms the VF approach, whereas the value function is symmetric in the wall environment, and the Euclidean distance between representations should be sufficient to learn it. This suggests that using a quasidistance still facilitates the training process.

It also appears that learning representations with both a prediction loss and an IQL-inspired one is less effective than using the latter loss alone. Using VCReg to promote diversity when learning with an IQL-inspired loss also seems to lead to poor planning performances.

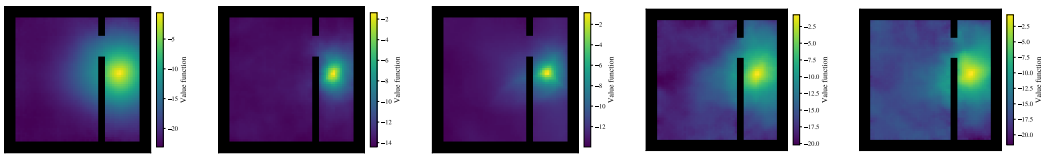
MPPI appears to be the most effective planning method. The CEM approach performs poorly, suggesting that its parameters may require better tuning.

The results obtained with the Wall_big dataset are better than those obtained with the Wall_small dataset. This may be due to the fact that a single trajectory explores more of the environment in the Wall_big dataset, and that the agent is more likely to collide with the wall. This suggests that using a hierarchy of representations with subsampled trajectories can be beneficial for planning.

Here are displayed visualizations of the associated value functions:



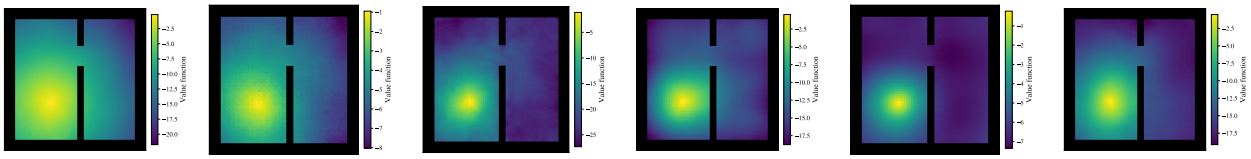
(a) Constrative (b) Regressive 1 (c) Regressive 2 (d) Pred_VCReg (e) Pred_EMA (f) VF



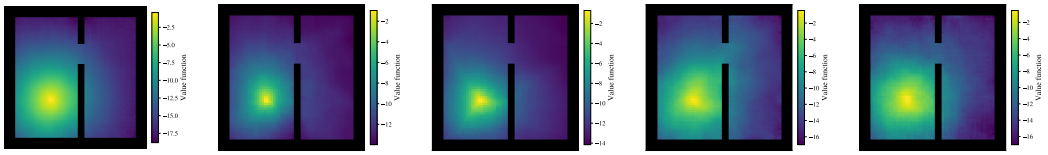
(g) VF_pred (h) VF_quasi (i) VF.quasi.pred (j) VF_VCReg (k) VF.vcr.pred

Figure 17: Visualization of the value function for the Wall_small dataset and a given goal

The value functions obtained for the Wall_big dataset tend to be better than the ones of the Wall_small dataset. They are more spread, and those of VF and VF_quasi correctly take the wall into account, without



(a) Constrative (b) Regressive 1 (c) Regressive 2 (d) Pred_VCReg (e) Pred_EMA (f) VF



(g) VF_pred (h) VF_quasi (i) VF.quasi.pred (j) VF_VCReg (k) VF.vcr.pred

Figure 18: Visualization of the value function for the Wall_big dataset and a given goal

crossing through it. This might explain the better results for these approaches. It is also interesting to note that the functions obtained for pred_VCReg and pred_EMA are similar to those computed with the value function-based approaches.

However, overall, the value functions are far from perfect and tend to ignore the wall. This is not surprising given the settings of the environment: the positions of the wall and door are random. Therefore, in most cases, the environment around the agent does not contain obstacles, making it harder for the model to account for the wall's influence on the value function.

The value functions still tend to encourage going through the door and some relevant information can be extracted from them, given the planning result. They also provide an explanation for the poor performances of the VF_VCReg approach: the value function appears to be very noisy for distant states.

In the case in which the wall is fixed and only the vertical position of the door is random, we obtain better looking value functions, and an almost perfect planning accuracy of 97% with the VF approach:

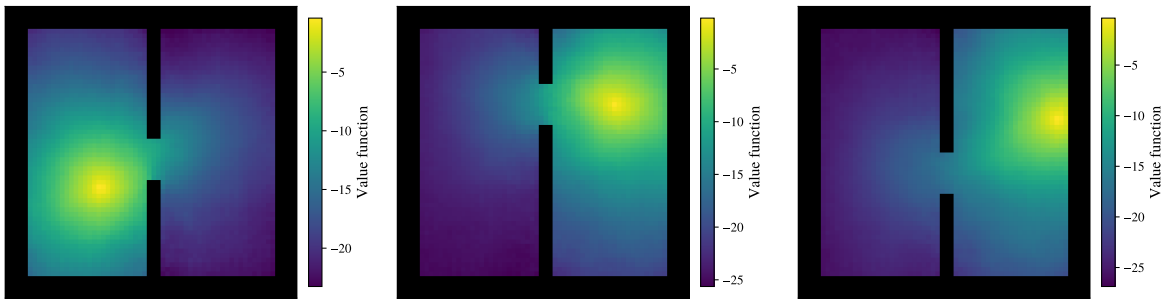


Figure 19: Visualization of the value function for an environment with fixed wall for different goals and doors

This suggests that increasing the expressivity of the networks used and the size of the training dataset could lead to similar results with the random wall settings.

To assess the impact of the network architecture, we evaluated the planning capabilities of a JEPA world model whose predictor is composed of several convolutional layers. In this case, the representations are three-dimensional rather than flat vectors. The architectures of the models are described in detail in Appendix 8.3. We experimented with representations of shape $8 \times 8 \times 8$ (matching the size of the flat representations

used previously) and of shape $8 \times 8 \times 16$. The results are as follows:

Type	MPPI
Contrastive	0.000
Regressive1	0.015
Regressive2	**0.450**
pred_VCReg	0.005
pred_EMA	0.335
VF	0.385
VF_pred	0.040
VF_quasi	0.095
VF_quasi_pred	0.405
VF_VCReg	0.430
VF_VCReg_pred	0.425

Table 3: Planning results for Wall_small with representations of shape $8 \times 8 \times 8$

Type	MPPI
Contrastive	0.015
Regressive1	0.030
Regressive2	0.590
pred_VCReg	0.030
pred_EMA	0.680
VF	0.640
VF_pred	0.735
VF_quasi	**0.750**
VF_quasi_pred	0.495
VF_VCReg	0.610
VF_VCReg_pred	0.615

Table 4: Planning results for Wall_small with representations of shape $8 \times 8 \times 16$

It appears that, when using such networks, larger representations are required to achieve results comparable to those obtained with flat representations. The pred_VCReg approach collapses in this settings, despite the use of VCReg, but the pred_EMA one yields good results.

Finally, one might hypothesize that representations learned using a prediction loss allow for better prediction accuracy, while those learned with an IQL-inspired approach lead to a planning cost. It is possible to take advantage of both by considering an intermediate approach. In this approach, two different representation spaces are learned: the first with a standard prediction loss, and the second with a value function loss using a second state encoder, in similar fashion with the hierarchical models. During planning, the first level is used to compute predictions, and the second level to compute the cost. We tested this approach using the VF loss for the second level on the Wall_small dataset. It did not improve planning results, with a planning accuracy of 0.595.

6.3.2 On the maze environment

We will now apply our different approaches to the maze environment. In this environment, the agent is a point mass with inertia. A state is composed of an observation image of the environment and of the velocity of the agent. The goal-conditioned value function is therefore not symmetric. We will compare the planning accuracies on 80 environments, and for two different ways of processing the proprioceptive information (velocities):

- Maze_sep: the approach used in [Sob+25], which consists in storing the information of the image of a state and of the velocities in separate parts of the representation vectors. At planning time, the cost used only leverages the representations of the images, discarding the ones of the velocities. In this case, the value function losses will only be applied to the image representations during training.
- Maze_mix: an approach that does not does not assume the ability to separate independent information, and that mixes the encodings of state images and velocities in the representations. At planning time, the goal is defined by the image corresponding to the desired agent location, with velocities set to zero.

We use the following parameters for the MPPI procedure: 500 initial perturbations are sampled from a Gaussian distribution with mean 0 and standard deviation 5, using a temperature parameter of $\lambda = 0.0025$. We use a planning horizon of 100, for a total of 200 planning steps.

Our results are the following:

Type	MPPI
Contrastive	0.50
Regressive1	0.46
Regressive2	0.30
pred_VCReg	0.54
pred_EMA	0.04
VF	0.49
VF_pred	0.49
VF_quasi	**0.63**
VF_quasi_pred	0.43
VF_VCReg	0.39
VF_VCReg_pred	0.39

Table 5: Planning results for Maze_sep

Type	MPPI
Contrastive	0.25
Regressive1	0.31
Regressive2	0.24
pred_VCReg	0.36
pred_EMA	0.39
VF	0.25
VF_pred	0.44
VF_quasi	**0.46**
VF_quasi_pred	**0.46**
VF_VCReg	0.2
VF_VCReg_pred	0.39

Table 6: Planning results for Maze_mix

The results in the Maze_sep settings are much better than those in the Maze_mix settings. This is not surprising, given that in the first settings we aided the model by keeping independent information separated in the representations.

In both settings, VF_quasi is the best performing approach, while VF performs worse than the standard pred_VCReg approach. This is probably linked to the fact that the theoretical goal-conditioned value function of the maze environment is not symmetric. It might also be explained by better generalization capabilities when using a prediction loss, since training layouts differ from the planning layouts.

It is worth noting that the hyperparameters of the IQL-inspired approaches have not been optimized for this environment and are the same as those used with the wall environment. However, the parameters of VCReg must be adjusted to achieve reasonable planning performance in the maze environment. In the wall environment, we computed VCReg only along the batch dimension, which was sufficient to prevent the collapse of representations. Nevertheless, when the environment varies significantly between trajectories in a batch, the representations can collapse: for a given layout of the environment, regardless of the agent's position, the representations may become identical to minimize the prediction loss. This occurs in the maze environment. To prevent it, VCReg must also be applied along the temporal dimension of the trajectories. However, doing so in the wall environment leads to poor results, likely because it makes minimizing the prediction loss more difficult.

In order to try to improve our results, we also evaluated our hierarchical approaches with the wall environment.

6.4 Impact of hierarchy

We now turn to the results obtained when using a hierarchical JEPA world model with two levels. We use a subsampling stride of 4 to train the second level of the hierarchy, and trajectories of length 8. We still use a subsampling stride of 1 and trajectories of length 16 for the first level of the hierarchy.

6.4.1 State hierarchy

We first evaluate the planning method we proposed for a state hierarchy on the Wall_small dataset.

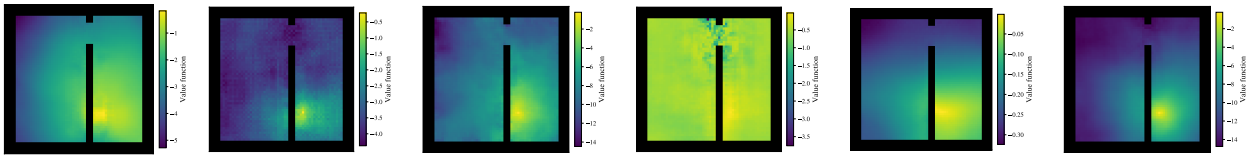
The planning accuracies obtained are largely similar to the ones achieved without a state hierarchy. Nevertheless, all planning results from IQL-inspired approaches show improvement, suggesting that the second level of the hierarchy provides useful information. Overall, the results are still not as good as those obtained with the Wall_big dataset.

They are displayed in the following table:

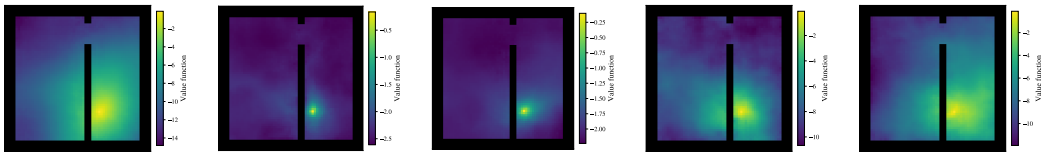
Type	MPPI	Diff
Contrastive	0.435	-0.050
Regressive1	0.610	0.070
Regressive2	0.585	-0.020
pred_VCReg	0.530	-0.020
pred_EMA	0.405	-0.050
VF	0.650	0.020
VF_pred	0.585	0.035
VF_quasi	**0.750**	0.040
VF_quasi_pred	0.590	-0.015
VF_VCReg	0.525	0.040
VF_VCReg_pred	0.510	0.040

Table 7: Planning results for Wall_small with a state hierarchy. Diff corresponds to the difference between these results and those obtained without a hierarchy

Here we display visualizations of the associated value functions:



(a) Constrative (b) Regressive 1 (c) Regressive 2 (d) Pred_VCReg (e) Pred_EMA (f) VF



(g) VF_pred (h) VF_quasi (i) VF.quasi.pred (j) VF_VCReg (k) VF.vcr.pred

Figure 20: Visualization of the second-level value function for a given goal in the Wall_small dataset

The value functions tend to be more spread out than at the first level of representations, as expected, with the notable exception of the quasimetric approaches and the first regressive one, which are instead more concentrated. The standard Pred_VCReg approach appears to have collapsed. It may be more complex to learn hierarchical state representations for this approach because it requires learning a state encoder, an action encoder, and a predictor, whereas only the state encoder matters for the others. Overall, all value functions are far from perfect, and seem to struggle to capture the influence of the wall.

These plots suggest that the approach could be further refined. The hyperparameters used for the different methods are likely not optimal for learning a second level of representations. In addition, the amount of data available for the second level is smaller than for the first one, since the trajectories are subsampled. Better results might be expected with a larger dataset.

6.4.2 Action hierarchy

We finally explore the capabilities of the action hierarchy planning approach on the Wall_small dataset. We use a planning horizon of 8 in the second representation space and 16 in the first representation space.

Here are our planning results without using the VAE-inspired approach:

Type	MPPI
Contrastive	0.020
Regressive1	0.160
Regressive2	0.480
pred_VCReg	0.020
pred_EMA	0.355
VF	**0.535**
VF_pred	0.465
VF_quasi	0.435
VF_quasi_pred	0.520
VF_VCReg	0.430
VF_VCReg_pred	0.490

Table 8: Planning results for Wall_small with an action hierarchy

These results are worse than those obtained without a hierarchy. To try to understand why, we visualize examples of plans output by the model during the MPC procedure:

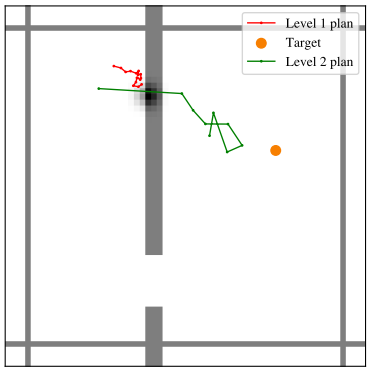


Figure 21: Visualization of a step of the hierarchical planning procedure

We obtained the planned trajectories by training a decoder for each representation space. These decoders output the agent's location corresponding to a given representation vector.

We observe that, in the second representation level, the planned trajectory completely ignores the wall, leaving the agent stuck behind it. We hypothesize that this type of erroneous planning arises from the optimization issues mentioned earlier and is responsible for the poor results. The optimized abstract actions correspond to unrealistic ones that allow the agent to cross the wall.

To address these issues, we trained a hierarchical JEPA model using the VAE-inspired approach. However, this led to even worse planning results. The constraints imposed on the action representations appear to prevent the predictor from achieving sufficient precision.

Further research is therefore necessary to properly optimize this approach.

7 Conclusion

In this study, we aimed to improve the planning capabilities of JEPA world models. To this end, we explored two main approaches. The first is to enhance the representations used for planning by learning them such that the Euclidean distance in the representation space approximates the negative of the goal-conditioned value function of the environment considered, associated with a goal-reaching cost. The second is to leverage a hierarchical structure, allowing for planning over longer time horizons and better encoding the relationships between temporally distant states of the environment.

In order to learn goal-conditioned value functions, we leveraged tools from the offline reinforcement learning setting, and more specifically implicit Q-learning approaches. These allowed us to define a loss on the representations learned by a JEPA model, enforcing our value function criterion. Since the goal-conditioned value function is not symmetric in general, we also attempted to learn it using a quasidistance in the representation space. We compared the effectiveness of these methods to more intuitive ones, as well as to the standard prediction-based JEPA approaches, on standard action planning tasks. Our results showed that the value function methods achieve better performance on these simple tasks.

However, it would be interesting to conduct further experiments with these different approaches. Prediction-based methods are expected to be more robust to randomness in non-deterministic environments and might enable learning more general representations than the other approaches tested. To properly evaluate these capabilities, it is necessary to test them on environments more complex than the ones we considered. An intermediate approach also appears promising in this context: prediction losses could be used to learn a first level of representations for prediction purposes, while IQL losses could train a second level dedicated to planning costs, and potentially specific to a given task.

We also proposed a method to learn a hierarchical JEPA model by subsampling training trajectories and encoding sequences of actions. We distinguished between two ways of using such a model for planning. The first is to use the hierarchy of state representations in the planning cost, so that it captures relationships between states at all scales. The second is to use the hierarchy of actions to plan over longer horizons. We tested these approaches with a two-level hierarchy. We only achieved slight improvements in planning results using the hierarchy of states. It is likely that the limited amount of data for training the second level negatively affected performance, and better results might be obtained with longer and more numerous trajectories.

We were unable to achieve good results by leveraging the hierarchy of actions. We identified a key issue that may explain this: when optimizing representations of actions, our procedure can produce adversarial examples that do not correspond to real actions. Several approaches could address this optimization problem. We proposed one inspired by variational autoencoders, but it would also be possible to regularize the action representations differently or use a reverse diffusion process to map any optimized action to the set of valid action representations. Further research in this direction could enable the effective use of hierarchical models for planning.

References

[Ass+23] Mahmoud Assran et al. *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture*. 2023. arXiv: 2301.08243 [cs.CV]. URL: https://arxiv.org/abs/2301.08243.

[BL24] Randall Balestriero and Yann LeCun. *Learning by Reconstruction Produces Uninformative Features For Perception*. 2024. arXiv: 2402.11337 [cs.CV]. URL: https://arxiv.org/abs/2402.11337.

[BPL22] Adrien Bardes, Jean Ponce, and Yann LeCun. *VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning*. 2022. arXiv: 2105.04906 [cs.CV]. URL: https://arxiv.org/abs/2105.04906.

[Bar+24] Adrien Bardes et al. *Revisiting Feature Prediction for Learning Visual Representations from Video*. 2024. arXiv: 2404.08471 [cs.CV]. URL: https://arxiv.org/abs/2404.08471.

[Din+25] Jingtao Ding et al. *Understanding World or Predicting Future? A Comprehensive Survey of World Models*. 2025. arXiv: 2411.14499 [cs.CL]. URL: https://arxiv.org/abs/2411.14499.

[Flo+21] Pete Florence et al. *Implicit Behavioral Cloning*. 2021. arXiv: 2109.00137 [cs.RO]. URL: https://arxiv.org/abs/2109.00137.

[Fu+21] Justin Fu et al. *D4RL: Datasets for Deep Data-Driven Reinforcement Learning*. 2021. arXiv: 2004.07219 [cs.LG]. URL: https://arxiv.org/abs/2004.07219.

[GPM89] Carlos E. García, David M. Prett, and Manfred Morari. "Model predictive control: Theory and practice—A survey". In: *Automatica* 25.3 (1989), pp. 335–348. ISSN: 0005-1098. DOI: https://doi.org/10.1016/0005-1098(89)90002-2. URL: https://www.sciencedirect.com/science/article/pii/0005109889900022.

[Gar+24] Quentin Garrido et al. *Learning and Leveraging World Models in Visual Representation Learning*. 2024. arXiv: 2403.00504 [cs.CV]. URL: https://arxiv.org/abs/2403.00504.

[Gar+25] Quentin Garrido et al. *Intuitive physics understanding emerges from self-supervised pretraining on natural videos*. 2025. arXiv: 2502.11831 [cs.CV]. URL: https://arxiv.org/abs/2502.11831.

[GBL23] Dibya Ghosh, Chethan Bhateja, and Sergey Levine. *Reinforcement Learning from Passive Data via Latent Intentions*. 2023. arXiv: 2304.04782 [cs.LG]. URL: https://arxiv.org/abs/2304.04782.

[GSS15] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. *Explaining and Harnessing Adversarial Examples*. 2015. arXiv: 1412.6572 [stat.ML]. URL: https://arxiv.org/abs/1412.6572.

[Gup+19] Abhishek Gupta et al. *Relay Policy Learning: Solving Long-Horizon Tasks via Imitation and Reinforcement Learning*. 2019. arXiv: 1910.11956 [cs.LG]. URL: https://arxiv.org/abs/1910.11956.

[Haf+24] Danijar Hafner et al. *Mastering Diverse Domains through World Models*. 2024. arXiv: 2301.04104 [cs.AI]. URL: https://arxiv.org/abs/2301.04104.

[Joh83] P. N. Johnson-Laird. *Mental models : towards a cognitive science of language, inference, and consciousness*. eng. Cognitive science series ; 6. Cambridge, Mass: Harvard University Press, 1983. ISBN: 0674568818.

[Jor+25] Armand Jordana et al. *Infinite-Horizon Value Function Approximation for Model Predictive Control*. 2025. arXiv: 2502.06760 [cs.RO]. URL: https://arxiv.org/abs/2502.06760.

[KW22] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: 1312.6114 [stat.ML]. URL: https://arxiv.org/abs/1312.6114.

[KNL21] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. *Offline Reinforcement Learning with Implicit Q-Learning*. 2021. arXiv: 2110.06169 [cs.LG]. URL: https://arxiv.org/abs/2110.06169.

[Law+25] Nathan P. Lawrence et al. *A view on learning robust goal-conditioned value functions: Interplay between RL and MPC*. 2025. arXiv: 2502.06996 [eess.SY]. URL: https://arxiv.org/abs/2502.06996.

[LC22] Yann LeCun and Courant. "A Path Towards Autonomous Machine Intelligence Version 0.9.2, 2022-06-27". In: 2022. URL: https://api.semanticscholar.org/CorpusID:251881108.

[Lev+20] Sergey Levine et al. *Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems*. 2020. arXiv: 2005.01643 [cs.LG]. URL: https://arxiv.org/abs/2005.01643.

[Ma+21] Xiaoteng Ma et al. *Offline Reinforcement Learning with Value-based Episodic Memory*. 2021. arXiv: 2110.09796 [cs.LG]. URL: https://arxiv.org/abs/2110.09796.

[Oqu+24] Maxime Oquab et al. *DINOv2: Learning Robust Visual Features without Supervision*. 2024. arXiv: 2304.07193 [cs.CV]. URL: https://arxiv.org/abs/2304.07193.

[PKL24] Seohong Park, Tobias Kreiman, and Sergey Levine. *Foundation Policies with Hilbert Representations*. 2024. arXiv: 2402.15567 [cs.LG]. URL: https://arxiv.org/abs/2402.15567.

[Par+24] Seohong Park et al. *HIQL: Offline Goal-Conditioned RL with Latent States as Actions*. 2024. arXiv: 2307.11949 [cs.LG]. URL: https://arxiv.org/abs/2307.11949.

[Sch+15] Tom Schaul et al. "Universal Value Function Approximators". In: *Proceedings of the 32nd International Conference on Machine Learning*. Ed. by Francis Bach and David Blei. Vol. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, July 2015, pp. 1312–1320. URL: https://proceedings.mlr.press/v37/schaul15.html.

[SKP15] Florian Schroff, Dmitry Kalenichenko, and James Philbin. "FaceNet: A unified embedding for face recognition and clustering". In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, June 2015, pp. 815–823. DOI: 10.1109/cvpr.2015.7298682. URL: http://dx.doi.org/10.1109/CVPR.2015.7298682.

[Shw+24] Ravid Shwartz-Ziv et al. *An Information-Theoretic Perspective on Variance-Invariance-Covariance Regularization*. 2024. arXiv: 2303.00633 [cs.IT]. URL: https://arxiv.org/abs/2303.00633.

[Sob+25] Vlad Sobal et al. *Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models*. 2025. arXiv: 2502.14819 [cs.LG]. URL: https://arxiv.org/abs/2502.14819.

[SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018. ISBN: 0262039249.

[WI22] Tongzhou Wang and Phillip Isola. "Improved Representation of Asymmetrical Distances with Interval Quasimetric Embeddings". In: *NeurIPS 2022 Workshop on Symmetry and Geometry in Neural Representations*. 2022. URL: https://openreview.net/forum?id=KRiST_rzkGl.

[Wan+23] Tongzhou Wang et al. *Optimal Goal-Reaching Reinforcement Learning via Quasimetric Learning*. 2023. arXiv: 2304.01203 [cs.LG]. URL: https://arxiv.org/abs/2304.01203.

[WAT15] Grady Williams, Andrew Aldrich, and Evangelos Theodorou. *Model Predictive Path Integral Control using Covariance Variable Importance Sampling*. 2015. arXiv: 1509.01149 [cs.SY]. URL: https://arxiv.org/abs/1509.01149.

[Xu+23] Haoran Xu et al. *A Policy-Guided Imitation Approach for Offline Reinforcement Learning*. 2023. arXiv: 2210.08323 [cs.LG]. URL: https://arxiv.org/abs/2210.08323.

[Zho+25] Gaoyue Zhou et al. *DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning*. 2025. arXiv: 2411.04983 [cs.RO]. URL: https://arxiv.org/abs/2411.04983.

8 Appendix

8.1 Proof of Theorem 1

In order to prove Theorem 1, let us prove the following lemma:

Lemma 2. *Let p be the density of a real random variable X with bounded support. Let $f : \mathbb{R}^2 \to \mathbb{R}$ be a continuous function, differentiable in its first variable. We suppose that $m, x \mapsto \frac{\partial f}{\partial m}(m, x)$ is continuous and that f is such that: $\forall m \in \mathbb{R}, f(m, m) = 0$.s Let us define: $F : m \mapsto \int_{-\infty}^{m} f(m, x)p(x)dx$.*

Then F is differentiable and:

$$\forall m \in \mathbb{R}, F'(m) = \int_{-\infty}^{m} \frac{\partial f}{\partial m}(m, x)p(x)dx \tag{40}$$

Proof of Lemma 2. Let X, p and f be defined as in the Lemma. Since it is a probability density, p is piecewise continuous. Let us define: $H : m, y \mapsto \int_{-\infty}^{y} f(m, x)p(x)\mathrm{d}x$.

We have:

- For all m in $\mathbb{R}$, the function $x \mapsto f(m, x)p(x)$ is integrable and piecewise continuous.
- The function $m \mapsto f(m, x)p(x)$ is differentiable for all x in $\mathbb{R}$.
- For all m in $\mathbb{R}$, the function $x \mapsto \frac{\partial f}{\partial m}(m, x)p(x)$ is piecewise continuous.
- For all x in $\mathbb{R}$, the function $m \mapsto \frac{\partial f}{\partial m}(m, x)p(x)$ is continuous.
- Since the support of X is bounded, there exists a constant c in $\mathbb{R}_+$ such that for all m, x in $\mathbb{R}^2$, the quantity $\left|\frac{\partial f}{\partial m}(m, x)p(x)\right|$ is smaller than $cp(x)$. The function $x \mapsto cp(x)$ is integrable.

Therefore, for every y in $\mathbb{R}$, we can apply the differentiation under the integral sign theorem to $m \mapsto H(m, y)$. It is differentiable and:

$$\forall (y, m) \in \mathbb{R}^2, \frac{\partial H}{\partial m}(m, y) = \int_{-\infty}^{y} \frac{\partial f}{\partial m}(m, x)p(x)\mathrm{d}x \tag{41}$$

Moreover, according to the fundamental theorem of integration, for every m in $\mathbb{R}$, $y \mapsto H(m, y)$ is differentiable from the right and from the left, and:

$$\forall (y, m) \in \mathbb{R}^2, \begin{cases} \frac{\partial H}{\partial y}(m, y^+) = \lim_{y_0 \to y^+} f(m, y_0)p(y_0) \\ \frac{\partial H}{\partial y}(m, y^-) = \lim_{y_0 \to y^-} f(m, y_0)p(y_0) \end{cases} \tag{42}$$

In particular, we have:

$$\forall m \in \mathbb{R}, \frac{\partial H}{\partial y}(m, m^+) = 0 = \frac{\partial H}{\partial y}(m, m^-) \tag{43}$$

Therefore, for all m in $\mathbb{R}$, $y \mapsto H(m, y)$ is differentiable at m, and its derivative is 0.

Therefore, for all m in $\mathbb{R}$, $F(m) = H(m, m)$ is differentiable and:

$$F'(m) = \int_{-\infty}^{m} \frac{\partial f}{\partial m}(m, x)p(x)\mathrm{d}x \tag{44}$$

□

Proof of Theorem 1. Let X be a random variable that satisfies the hypotheses of the theorem. Let x^+ be the supremum of its support, and x^- its infimum. Let $\tau \in [0, 1]$. Let us derive: $f_\tau : m \mapsto \mathbb{E}_{x \sim X}(L_\tau^2(x - m))$. This function is twice differentiable according to Lemma 2, and:

$$\forall m \in \mathbb{R}, f'_\tau(m) = \int_{-\infty}^{m} -2(1 - \tau)(x - m)p(x)\mathrm{d}x + \int_{m}^{+\infty} -2\tau(x - m)p(x)\mathrm{d}x \tag{45}$$

where p is the density of X. We also have:

$$\forall m \in \mathbb{R}, f''_\tau(m) = \int_{-\infty}^{m} 2(1-\tau)p(x)\mathrm{d}x + \int_{m}^{+\infty} 2\tau p(x)\mathrm{d}x \tag{46}$$

$$> 0 \text{ if } \tau \in]0,1[\tag{47}$$

Therefore f_τ is strictly convex if $\tau \in]0,1[$, and the existence and uniqueness of m_τ are proved for all τ in $]0,1[$.

Additionally, if x^+ is the supremum of the support of X, we have:

$$f'_\tau(x^+) = \int_{-\infty}^{x^+} -2(1-\tau)(x-x^+)p(x)\mathrm{d}x + \int_{x^+}^{+\infty} -2\tau(x-x^+)p(x)\mathrm{d}x \tag{48}$$

$$= \int_{-\infty}^{x^+} -2(1-\tau)(x-x^+)p(x)\mathrm{d}x \tag{49}$$

$$\geqslant 0 \tag{50}$$

Therefore f_τ is non decreasing at x^+ and $m_\tau \leqslant x^+$ for all τ in $]0,1[$.

Let $(\tau_1, \tau_2) \in]0,1[^2$ such that $\tau_1 \leqslant \tau_2$. We have $f'_{\tau_1}(m_{\tau_1}) = 0$ and:

$$f'_{\tau_2}(m_{\tau_1}) = f'_{\tau_2}(m_{\tau_1}) - f'_{\tau_1}(m_{\tau_1}) \tag{51}$$

$$= \int_{-\infty}^{m_{\tau_1}} -2(\tau_1-\tau_2)(x-m_{\tau_1})p(x)\mathrm{d}x + \int_{m_{\tau_1}}^{+\infty} -2(\tau_2-\tau_1)(x-m_{\tau_1})p(x)\mathrm{d}x \tag{52}$$

$$\leqslant 0 \tag{53}$$

Therefore f_{τ_2} is non increasing at m_{τ_1} and $m_{\tau_2} \geqslant m_{\tau_1}$. The function $\tau \mapsto m_\tau$ is non decreasing on $]0,1[$ and is bounded above by x^+ on $]0,1[$. Therefore it converges and $\lim_{\tau\to 1}(m_\tau) \leqslant x^+$.

Let $m \in]x^-, x^+[$. We have:

$$f'_0(m) = \int_{-\infty}^{m} -2(x-m)p(x)\mathrm{d}x > 0 \qquad \text{and} \qquad f'_1(m) = \int_{m}^{+\infty} -2(x-m)p(x)\mathrm{d}x < 0 \tag{54}$$

The strict inequalities come from the fact that $x^- < m < x^+$ and that x^- and x^+ are the infimum and supremum of the support of X.
The function $\tau \mapsto f'_\tau(m)$ is continuous. Therefore, by the intermediate value theorem:

$$\exists \tau^\star \in]0,1[, f'_{\tau^\star}(m) = 0 \tag{55}$$

Therefore: $\forall m \in]x^-, x^+[, \exists \tau^\star \in]0,1[, m = m_{\tau^\star}$.
Moreover: $\forall \tau_0 \in]0,1[, \lim_{\tau\to 1}(m_\tau) \geqslant m_{\tau_0}$, and therefore: $\lim_{\tau\to 1}(m_\tau) \geqslant x^+$.

We can conclude that:

$$\lim_{\tau\to 1}(m_\tau) = x^+ \tag{56}$$

□

8.2 Properties of goal-conditioned value functions

Goal-conditioned value functions for reaching goals have some interesting properties that make their learning meaningful.

Theorem 3. ?? *We define a reaching cost:* $C : \begin{cases} \mathrm{S}^2 \times \mathrm{A} \to \mathbb{R}_+ \\ (s,g,a) \mapsto \mathbb{1}(s \neq g) \end{cases}$.
Let d *denotes the dynamics of the environment considered. We suppose that there exists an action* a_0 *such that:* $\forall s \in \mathrm{S}, d(s, a_0) = s$.
We also suppose that: $\exists c \in \mathbb{R}_+^*, \forall (s,a) \in \mathrm{S} \times \mathrm{A}, \|d(s,a) - s\| \leqslant c$.

Let $V^\star$ *be the goal-conditioned value function associated to* C*. Then* $V^\star$ *is such that:*

$$\forall (s,g) \in \mathrm{S}^2, -V^\star(x,g) \geqslant \frac{1}{c}\|s-g\|_2 \tag{57}$$

Proof of Theorem **??**. Let us define $V^\star$ and the environment as in the theorem. Let $(s,g) \in \mathrm{S}^2$.

If it is not possible to reach g from s, then $-V^\star(s,g) = +\infty$, and the inequality is true.

Otherwise, let us fix the minimum number n of actions necessary to reach g, the optimal sequence of actions $(a_i)_{i \in [\![0,n-1]\!]}$, and the associated sequence of states $(s_i)_{i \in [\![0,n]\!]}$. We have:

$$\|s-g\|_2 \leqslant \sum_{t=0}^{n-1} \|d(s_i,a_i) - s_i\|_2 \leqslant c\,n = -c\,V^\star(s,g) \tag{58}$$

According to the triangle inequality. The theorem is therefore proved. □

Theorem 4. *Let* $d \in \mathbb{N}$, $\epsilon > 0$, $f : \mathbb{R}^d \to \mathbb{R} \cup \{+\infty\}$ *arbitrary and* $g : \mathbb{R}^d \to \mathbb{R}$ *continuous.*
We define $\mathcal{D} := \{x \in \mathbb{R}^d \mid f(x) < +\infty\}$.
We suppose that:

$$\forall x \in \mathcal{D}, \|f(x) - g(x)\| \;\leq\; \varepsilon \quad et \quad \forall x \in \mathbb{R}^d \setminus \mathcal{D}, g(x) > \inf(g_{|\mathcal{D}}) \tag{59}$$

Let us suppose that there exist a $x_0 \in \mathbb{R}^d$ *such that:*

$$\exists c \in \mathbb{R}_+^\star, \forall x \in \mathbb{R}^d, f(x) \geqslant f(x_0) + c\|x - x_0\|_2 \tag{60}$$

Then f *attains its global minimum, and* x_0 *is its unique minimizer. Moreover,* g *attains its global minimum, and every minimizer* x_1 *of* g *is such that:*

$$\|x_1 - x_0\|_2 \leqslant \frac{2\varepsilon}{c} \tag{61}$$

Proof of Theorem 4. Let f, g two functions which satisfies the hypotheses of the theorem. We define $\mathcal{D}$ as in the theorem.

Given the inequality satisfies by f, we have: $\forall x \in \mathbb{R}^d, x \neq x_0 \Rightarrow f(x) > f(x_0)$.
The function f attains its global minimum, and x_0 is its unique minimizer.

We have $x_0 \in \mathcal{D}$. Let us define $\mathrm{C} := \{y \in \mathbb{R}^d \mid c\|x_0 - y\|_2 \leqslant 2\epsilon\}$. The function g attains its minimum in the compact C at a $x_1 \in \mathrm{C}$. Without loss of generality, let us suppose that x_1 is unique. Given the conditions, $g(x_1)$ is a global minimum. Indeed, let $y \in \mathbb{R}^d \setminus \mathrm{C}$. If $y \in \mathcal{D}$, we have

$$g(y) \geqslant f(y) - \epsilon \geqslant f(x_0) + \epsilon \geqslant g(x_0) \geqslant g(x_1) \tag{62}$$

Otherwise, we have $g(y) > \inf(g_{|\mathcal{D}}) \geqslant g(x_1)$.
And, since x_1 is in C, we have: $\|x_0 - x_1\|_2 \leqslant \frac{2\epsilon}{c}$.

□

Let $g \in S$, and $V^\star$ be the goal-conditioned value function associated to our standard goal-reaching cost. The function $-V^\star(\cdot, g)$ satisfies the hypothesis 60 of theorem 4 ($V^\star(g,g) = 0$). When learning it with a neural network, this means that if the rest of the conditions of theorem 4 are satisfied, we have guaranties about the quality of the minimum of the learned function. This is important because the planning procedure attempts to reach this minimum.

8.3 Architectures

Here we provide more details about the architectures we used.
When using flat representations, we use for level 1:

- a state encoder composed of 3 ResnetStacks and a linear layer, which output of vector of size 512. A ResnetStack is a combination of an initial convolution layer, a max pooling operation (with kernel size 3, stride 2 and padding 1), and 4 convolutions with residual connections every 2 layers. Each of these last convolution is followed by a ReLU activation. The number of channels used for the ResnetStacks are respectively 16, 32 and 32. All convolutions have a kernel 3×3.
- an action encoder equal to the identity
- a predictor composed of a linear layer turning the action into a vector of dimension 512, and 4 linear layers, all followed by ReLU activations, except for the last one. All their output dimensions are 512.

For level 2, we use:

- a state encoder composed of 3 linear layers, all followed by ReLU activations, except for the last one. All their output dimensions are 512.
- an action encoder composed of 4 linear layers, all followed by ReLU activations, except for the last one. All their output dimensions are 16 (except for the last one, which might be 32 if we use the VAE approach).
- a predictor composed of a linear layer turning the action into a vector of dimension 512, and 3 linear layers, all followed by ReLU activations, except for the last one. All their output dimensions are 512.

When using convolutional representations, we use for level 1:

- a state encoder composed of 3 ResnetStacks and a convolutional layer with kernel size 1 and 16 output channels. A ResnetStack is a combination of an initial convolution layer, 4 convolutions with residual connections every 2 layers, and a max pooling operation with kernel size 3, stride 2 and padding 1. The number of channels used for the ResnetStacks are respectively 16, 32 and 32. All convolutions have a kernel 3×3.
- an action encoder equal to the identity
- a predictor composed of a linear layer turning the action into a vector of dimension 16, and 3 convolutional layers, all followed by ReLU activations, except for the last one. They all have 16 output channels. The action representation is repeated and concatenated with the state representation along the channel dimension.

Convolutional representations are flattened before being input into the losses used for training.